List of exercises for courses "Statistics in Health Sciences" (SHS) and "Cross-sectional Data Analysis" (CDA)

Version 1.3. Updated on Wednesday, September 23, 2026 at 12:04:10

Jose Barrera
jose.barrera@uab.cat

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 4.0 International"

About this list of exercises

Despite these exercises are not to be delivered, students should solve them during the course. This list of exercises must be considered as personal work that each student must plan according to his or her agenda. Some of these exercises will be solved by the teacher during practice sessions.

Exercises for the course "Statistics in Health Sciences" are indicated with (SHS) while exercises for the course "Cross-sectional Data Analysis" are indicated with (CDA).

Exercises labelled with the symbol * are considered especially difficult. Motivated students are welcome to try although procedures and tools needed to solve them are beyond the scope of the course.

Contents

1 Getting familiar with scientific papers (SHS/CDA) 1

1.1 PubMed . . . 1

1.2 Reading a paper . . . 1

2 Study design (SHS/CDA) 3

2.1 Setting the study design in hypothetical studies . . . 3

2.2 Identifying study design in scientific papers . . . 4

3 Study design and contingency table analysis (SHS/CDA) 5

4 Measurement error and missing data (SHS) 7

4.1 Estimating the number of complete cases in a dataset . . . 7

4.2 Measurement error, missing data and bias* . . . 7

5 Multiple imputation for linear regression analysis (solved) (SHS/CDA) 9

5.1 Introduction . . . 9

5.2 The `nhanes` data . . . 9

5.3 Missingness exploration . . . 11

5.4 Data analysis . . . 12

5.4.1 Complete cases analysis . . . 13

5.4.2 Simple mean imputation . . . 14

5.4.3 Multiple imputation . . . 16

5.4.3.1 Illustration . . . 16

5.4.3.2 Comparison of results . . . 23

6 Multiple imputation for clustering analysis (solved) (SHS/CDA) 25

6.1 Introduction . . . 25

6.2 Data . . . 25

6.3 k-means clustering with variable selection in the presence of missing data . . . 28

6.3.1 Data preparation for miclust package . . . 28
6.3.2 Clustering procedure . . . 30
6.3.3 Clustering results . . . 30

7 Measures of the disease **35**
7.1 Prevalence (SHS/CDA) . . . 35
7.1.1 Estimating prevalences . . . 35
7.1.2 Comparing prevalences . . . 35
7.1.3 Exact hypothesis test for a prevalence . . . 36
7.2 Incidence rate (SHS) . . . 37

8 Analysis of the contingency table for crude association assessment (SHS/CDA) **39**

9 Estimating OR in a matched case-control study (SHS) **41**
9.1 Introduction . . . 41
9.2 Review of (unmatched) case-control design . . . 41
9.2.1 Illustrative example (unmatched) . . . 42
9.3 Matched case-control studies (1:1 matching) . . . 42
9.3.1 Types of pairs . . . 42
9.3.2 Estimation of the OR . . . 42
9.3.3 McNemar test . . . 43
9.3.4 Comments . . . 43
9.3.5 Exercise . . . 44
9.4 Consequences of ignoring matching in the analysis: Confounding control . . . 45
9.5 Alternative when matching by design is unfeasible . . . 46

10 Estimating risks under different sampling designs* (SHS) **47**

11 The logistic regression model (CDA) **51**
11.1 Data simulation . . . 51
11.2 Fitting the logistic regression model . . . 55
11.3 Assessing the quality of the simulation study . . . 57

12 R programming* (SHS/CDA) **59**
12.1 Correction of sleep detection . . . 59
12.1.1 Writing R functions . . . 59
12.1.2 Exploring computation time . . . 60
12.2 Fast categorization in a particular case . . . 60

12.2.1 Writing R functions 60
12.2.2 Comparing computation times 61

Bibliography **63**

A R code for Section 7.1.1 **65**

B Estimation of OR under a 1:1 matching case-control design **67**

C R code for Chapter 10 **69**

Chapter 1

Getting familiar with scientific papers (SHS/CDA)

1.1 PubMed

Use the advanced search of PubMed to find a paper, published not before 2009, that assessed the association between the cognitive function and overweight in children.

1. Write the syntax you have used to find the paper.
2. Write the following information about the paper:
   - Title
   - Names of the authors and their affiliations
   - Date of publication
   - Name of the journal in which the paper was published

1.2 Reading a paper

Regarding the paper on the study by Guxens et al. [1], answer the following questions:

1. What were the objectives of the study?
2. What variables did the authors considered? How and when did those variables been measured?
3. Briefly describe the sample of the study.
4. What variables were considered as exposures?
5. What variable was considered as the response?
6. What variables were considered as confounders?
7. Which statistical method was used to deal with missing data? Briefly describe such method.
8. What measures of association were used?

Chapter 2

Study design (SHS/CDA)

2.1 Setting the study design in hypothetical studies

For each of the following hypothetical studies, identify the exposure, the response variable and the study design. In those cases in which the study design is not set, decide what would be the most appropriate one.

1. A study to assess association between eoily fish consumption in women during pregnancy and the risk of congenital heart defects in their baby. Specifically, the researcher recruited a group of mothers of babies with congenital heart defects and another group of mothers of babies without congenital heart defects. Then, each of the recruited mothers was interviewed about different aspects of their diet during pregnancy, including oily fish consumption.

2. A study to assess the association between consumption of processed red meat and coronary heart disease compared the average consumption per person in each of 50 countries with the incidence rate of coronary heart disease in each of the same 50 countries. All data were obtained from existing registries provided by consumer and public health agencies.

3. We want to carry out a study to assess the short-term association between walnut consumption and cognitive function in primary school children. Specifically, the research team hypothesizes that the daily consumption of 50 g of walnuts could have favorable effects on cognitive function in the short term (months). 23 schools will participate in the study with a total of 879 students participating in the study. To decide the study design, take into account the characteristics of the exposure and the fact that it is desirable to control as much as possible for potential confounding by other variables that cannot be measured in the study.

4. A study is intended to examine the possible short-term effects of daily air pollution levels in a given city on the health of its inhabitants, specifically on the number of daily hospitalizations of city residents. The aim is to take into account the lagged effect phenomenon.

5. In a study to assess the association between alcohol consumption and breast cancer, the research group recruits a group of women over 50 years of age who suffer or have suffered from breast cancer. To each of these women, the research group gives a health questionnaire where they collect information regarding alcohol consumption and other variables related to lifestyle and socioeconomic conditions. Once this information is obtained, for each woman with breast cancer recruited, the research group randomly recruits, from a pre-existing database of healthy women (i.e., who have not suffered from breast cancer), a woman over 50 years of age and with a similar lifestyle and socioeconomic profile. To each of these women, the research group gives the same health questionnaire. Based on all the

information collected among all the participating women, the association between alcohol consumption and breast cancer is analysed.

6. In a study on the association between coffee consumption and performance on a memory test, a group of volunteer participants were recruited. The researcher randomly assigned half of the individuals to drink coffee one hour before taking the memory test and the other half to drink a caffeine-free liquid preparation one hour before taking the memory test.

7. A study recruits a group of college graduates, all of whom have healthy cholesterol levels. At the time of recruitment, participants provide a sample of their blood that is immediately analysed. Participants then complete a health questionnaire about their lifestyle habits, including their physical activity. Participants are followed over time, with medical check-ups every two years that include repeating the health questionnaire and a blood test to measure cholesterol levels. At the end of the study, researchers analyse the data obtained to compare, in terms of average cholesterol levels, individuals who have been classified as sedentary with those who have not.

8. A researcher recruits a group of healthy individuals and distributes questionnaires to collect information about physical activity frequency and intensity and body mass index (BMI). The researcher then assesses the association between physical activity pattern and BMI.

9. A researcher uses a database of medical records to identify a group of retired construction workers. She reviews the medical records of each of these individuals to track both their occupational sun exposure throughout their working lives and melanoma incidence. Then, she compares the subset of individuals that developed melanoma with the subset of individuals that did not, in terms of the estimated accumulated ultraviolet radiation exposure during the working live.

2.2 Identifying study design in scientific papers

For each of the papers on the studies by Steindorf et al.[2], Lewis et al.[3] and Hoddinott et al.[4]:

1. Identify the study design.

2. Identify the exposure.

3. Identify the response variable.

4. Briefly summarize the results found in the study.

Chapter 3

Study design and contingency table analysis (SHS/CDA)

When assessing the crude association between a categorical exposure and a categorical response with the analysis of the contingency table, the joint probability distribution of the cells counts in the table depends on the study design that provides the data to be analysed. For each of the following hypothetical studies, characterize the joint probability distribution of the cells counts in the corresponding contingency table.

1. Case-control study to evaluate the association between liver problems ("control", "non-serious problems" or "serious problems") and alcohol consumption ("non-consumption", "sporadic consumption", "moderate consumption" or "heavy consumption").

2. Cohort study to evaluate the association between liver problems ("control", "non-serious problems" or "serious problems") and alcohol consumption ("non-consumption", "sporadic consumption", "moderate consumption" or "heavy consumption").

3. Cross-sectional study to evaluate the association between liver problems ("control", "non-serious problems" or "serious problems") and alcohol consumption ("non-consumption", "sporadic consumption", "moderate consumption" or "heavy consumption").

Chapter 4

Measurement error and missing data (SHS)

4.1 Estimating the number of complete cases in a dataset

1. A cross-sectional study is being designed based on a survey that will be given to n participants. The results of the survey will be transferred into a dataset generating 32 variables. It is expected that some survey responses will result in missing data in the dataset and the pattern of missing values this will cause in the dataset is assumed to be MCAR. Furthermore, it is assumed that, for 12 variables, each have a probability of missing data equal to 0.05; for 9 other variables, each have a probability of missing data equal to 0.03; and each of the remaining variables has a probability of missing data equal to 0.01. Based on these assumptions, calculate the minimum value of n needed if the researcher wants the expected number of complete cases in the dataset to be no less than 2000.

2. We have daily time series of $PM_{2.5}$ levels recorded at 8 measuring stations located at different points in a given city. Thus, for a given day, we have 8 $PM_{2.5}$ values (one for each station). With these 8 daily time series, we want to construct a new daily time series that represents the average daily levels of $PM_{2.5}$ in the city. In this new time series, the value for each day is calculated as the arithmetic mean of the 8 values recorded at the different stations for that day. To calculate these daily averages, the study protocol indicates that, if a station has a missing value (i.e., at that station the $PM_{2.5}$ level could not be recorded on that day), the average value for that day will be calculated using the values available for that day at the other stations (i.e., considering only the stations that recorded the $PM_{2.5}$ level on that day). However, in the case of missing value for the day in question in more than one station, the corresponding average value for the new time series will also be lost (i.e., the average will be a missing value if there is no recording of the $PM_{2.5}$ level in at least 7 stations for the given day). After exploring the historical data of the 8 stations, it is assumed that 5 of the stations never fail (i.e., they record the $PM_{2.5}$ value every day); two other stations fail each day with probability 0.1 each and independently; and the remaining station fails each day with probability 0.3, independently of the other stations. It is also assumed that the missing data caused by these failure probabilities follow an MCAR pattern. Calculate the expected percentage of days on which the new time series of daily mean values will have a missing value.

4.2 Measurement error, missing data and bias*

In a hypothetical study, we are going to select a random sample of size n in a given population. We will measure two variables, X and Y, on each of the individuals in the sample. We assume that $X \sim \mathcal{N}(\mu, \sigma)$ and $Y \sim \text{Bern}(\pi)$. We also assume that X and Y are independent. In addition, we know that we are going to have the following issues during the sampling process:

- Y can be missing with probability m_Y, which is the same for all individuals.
- X will be measured by a device which is perfectly calibrated, that is to say there is no measurement error. However, the device does not work properly for extreme values of X. Specifically, if the distance between the true value of X and μ is greater than $a\sigma$ units, where $a > 0$, then the device sometimes results in an error message which, in practice, is treated as a missing datum. The probability of such an error is m_X, which is a constant.

1. Let π_c be the probability that an individual randomly chosen from the population to be part of the sample is a complete case (i.e., we obtain no missing data for that individual). Find π_c as a function of the parameters μ, σ, π, m_Y, a and m_X.
2. Provide a graphical representation of π_c.
3. Find the expected percent of complete cases in the sample in the two following scenarios:
   (a) $m_Y = 0.1$, $m_X = 0.1$, and $a = 2$.
   (b) $m_Y = 0.1$, $m_X = 0.9$, and $a = 3$.

   Those results seem to be independent of the values of m_X and a. Why?
4. Let $\hat{\mu}$ and $\hat{\sigma}^2$ the mean and variance estimates of X, respectively, using data in the sample (ignoring missing values), and let $\psi := \hat{\mu} - \mu$ and $\phi := \frac{\hat{\sigma}^2}{\sigma^2}$. Without performing any calculation:
   (a) Prove that both ψ and ϕ are independent on both μ and σ.
   (b) Prove that, in general, $\psi = 0$ and $\phi < 1$.
5. Perform a simulation study to explore the values of ψ and ϕ as a function of explored values of a and m_X. You should proceed as follows:
   - Consider the values $m_X = 0.1, 0.2, 0.3, \ldots, 0.8, 0.9$ and $a = 0.5, 1, 1.5, 2, \ldots, 3.5, 4$.
   - For each combination of the values of a and m_X above, simulate a sampled vector of length $n = 10000$ from the variable X.
   - Compute the values of ψ and ϕ in the previous sample.
   - Perform steps 2 and 3 $M = 1000$ times so you obtain M values of ψ and ϕ.
   - Compute the mean and also percentiles 2.5th and 97.5th of both ψ and ϕ across the M previous simulations.
   - Finally, show graphically the behavior of biases ψ and ϕ as a function of a and m_X.

Chapter 5

Multiple imputation for linear regression analysis (solved) (SHS/CDA)

5.1 Introduction

In this exercise, we well see an example of dealing with missing data before performing a linear regression analysis. Specifically, we will consider three scenarios:

- Complete cases analysis.
- Simple mean imputation.
- Multiple imputation.

Review the slides of the course "Applications of Statistics to Health Sciences" to get familiar with the three techniques above and their implications on the posterior data analysis, in terms of bias and variance.

5.2 The `nhanes` data

Consider `nhanes2` data from the `mice` package:

```
# install.packages("mice") # if you need to install the 'mice' package
library(mice)
data(nhanes2)
nhanes2
```

```
##      age  bmi  hyp chl
## 1  20-39   NA <NA>  NA
## 2  40-59 22.7   no 187
## 3  20-39   NA   no 187
## 4  60-99   NA <NA>  NA
## 5  20-39 20.4   no 113
## 6  60-99   NA <NA> 184
## 7  20-39 22.5   no 118
## 8  20-39 30.1   no 187
## 9  40-59 22.0   no 238
## 10 40-59   NA <NA>  NA
## 11 20-39   NA <NA>  NA
```

```
## 12 40-59   NA <NA>  NA
## 13 60-99 21.7   no 206
## 14 40-59 28.7  yes 204
## 15 20-39 29.6   no  NA
## 16 20-39   NA <NA>  NA
## 17 60-99 27.2  yes 284
## 18 40-59 26.3  yes 199
## 19 20-39 35.3   no 218
## 20 60-99 25.5  yes  NA
## 21 20-39   NA <NA>  NA
## 22 20-39 33.2   no 229
## 23 20-39 27.5   no 131
## 24 60-99 24.9   no  NA
## 25 40-59 27.4   no 186
```

```
summary(nhanes2)
```

```
##     age          bmi         hyp          chl
##  20-39:12   Min.   :20.40   no :13   Min.   :113.0
##  40-59: 7   1st Qu.:22.65   yes: 4   1st Qu.:185.0
##  60-99: 6   Median :26.75   NAs: 8   Median :187.0
##             Mean   :26.56            Mean   :191.4
##             3rd Qu.:28.93            3rd Qu.:212.0
##             Max.   :35.30            Max.   :284.0
##             NAs    :9                NAs    :10
```

Read the data help to get familiar with them:

```
?nhanes2
```

Now, we include a categorical version of the body mass index (BMI). According to the WHO, BMI categories are:

$$\text{BMI category} = \begin{cases} \text{Underweight}, & \text{if BMI} < 18.5 \\ \text{Normal}, & \text{if BMI} \in [18.5, 25) \\ \text{Overweight}, & \text{if BMI} \in [25, 30) \\ \text{Obese}, & \text{if BMI} \geqslant 30 \end{cases}$$

Note that the minimum BMI in the dataset is 20.4, so there is no underweight individual. Hence, we could ignore that category when creating the categorical BMI, using `droplevels()`. However, we need to be careful since, in the imputation process we will perform later, we could get imputed values of BMI lower than 18.5, which would be a underweight individual. Hence, we can proceed as follow: 1) we will work ignoring the underweight category and, 2) after the imputation process is finished, we will check the possible presence of values imputed as underweight.

To facilitate the work, we write and use the following `getBMIgroup()` function, which internally uses `droplevels()`:

```
getBMIgroup <- function(bmi) {  # numerical BMI
  ### categorization of BMI according to WHO:
  # underweight if BMI < 18.5
  # normal      if BMI in [18.5, 25)
  # overweight  if BMI in [25, 30)
  # obese       if BMI >= 30
  bmibreaks <- c(0, 18.5, 25, 30, Inf)
  bmilevels <- c("underweight", "normal", "overweight", "obese")
  res <- cut(bmi,
             breaks = bmibreaks,
             labels = bmilevels,
             include.lowest = TRUE,
             right = FALSE)
  res <- droplevels(res)
  return(res)
}
```

Now, we create `bmig` as the BMI category:

```
nhanes2$bmig <-  getBMIgroup(nhanes2$bmi)

# check:
library(doBy)
summaryBy(bmi ~ bmig, FUN = c(min, max), data = nhanes2)

##         bmig bmi.min bmi.max
## 1     normal    20.4    24.9
## 2 overweight    25.5    29.6
## 3      obese    30.1    35.3
## 4       <NA>      NA      NA

summary(nhanes2)

##     age          bmi         hyp          chl               bmig
##  20-39:12   Min.   :20.40   no :13   Min.   :113.0   normal    :6
##  40-59: 7   1st Qu.:22.65   yes: 4   1st Qu.:185.0   overweight:7
##  60-99: 6   Median :26.75   NAs: 8   Median :187.0   obese     :3
##             Mean   :26.56            Mean   :191.4   NAs       :9
##             3rd Qu.:28.93            3rd Qu.:212.0
##             Max.   :35.30            Max.   :284.0
##             NAs    :9                NAs    :10
```

5.3 Missingness exploration

Percentage of missing observations for each variable, sorted by decreasing missingness, and percentage of complete cases:

```
### variables missingnes (%):
(miss100 <- sort(100 * colMeans(is.na(nhanes2)), decreasing = TRUE))

##  chl  bmi bmig  hyp  age
##   40   36   36   32    0

### complete cases (%):
(cc100 <- 100 * mean(complete.cases(nhanes2)))

## [1] 52
```

We see that only `age` is complete and only 52% of cases are complete.

The `md.pattern()` function in the mice package provides further information, both numerical and graphical (see Figure 5.1)) about the missingness pattern:

```
mice::md.pattern(nhanes2, rotate.names = TRUE)
```

```
##    age hyp bmi bmig chl
## 13   1   1   1    1   1  0
## 3    1   1   1    1   0  1
## 1    1   1   0    0   1  2
## 1    1   0   0    0   1  3
## 7    1   0   0    0   0  4
##      0   8   9    9  10 36
```

("1" means observed; "0" means missing)

5.4 Data analysis

Suppose we are interested in estimate a possible linear association between BMI (as continuous) and cholesterol adjusting by age and hypertension. Figure 5.2 is a visualization of the crude association between the variables of interest.

Specifically, we are interested in estimate the β coefficient associated to the BMI in a linear regression model with cholesterol as the outcome (i.e., response variable) and three explanatory variables: BMI, as the exposure of interest, and age and hypertension, as potential confounders. To do this in an automatic way, we first create a function which performs the required analysis for a given dataset with the same structure than `nhanes2`:

```
getBetaBMI <- function(mydata = nhanes2) {
  mod <- lm(chl ~ bmi + hyp + age, data = mydata)
  coefs <- summary(mod)$coefficients
  ci <- confint(mod)
  coefs <- cbind(coefs, ci)
  beta <- coefs["bmi", c("Estimate", "Std. Error", "Pr(>|t|)", "2.5 %", "97.5 %")]
  return(beta)
}
```

Then, we will use such a function in the three scenarios proposed in Section 5.1.

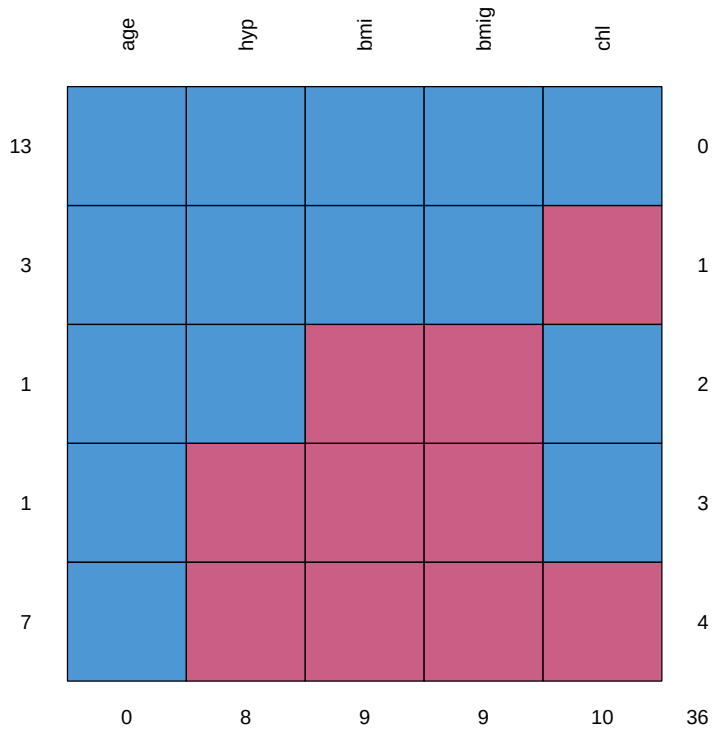


Figure 5.1: Missingness pattern in `nhanes2` data (obtained with the `md.pattern()` function). Blue means observed while maroon means missing. Numbers in the left column represent the number of observations with the corresponding row pattern (e.g., 13 complete cases, or 3 individuals missing only `chl`). Numbers in the right column represent the number of missing values in the corresponding row pattern. Numbers in the bottom row represent the number of individuals with missing value in the corresponding column variable (e.g., `age` is completely observed, or `hyp` value is missed for 8 individuals).

5.4.1 Complete cases analysis

In this case, we just need to pass `nhanes2` to our function `getBetaBMI()`, because the default setting of `lm()` ignores observations with missing data. We save results in the object `effectCC`:

```
effectCC <- getBetaBMI(nhanes2)
effectCC
```

```
##    Estimate  Std. Error    Pr(>|t|)       2.5 %      97.5 %
##  7.16011893  2.20089860  0.01164394  2.08483765 12.23540020
```

Interpretation: Under the fitted model using only complete cases, we estimate that each 1 kg/m^2 increase in BMI is associated with a 7.2 mg/dl ($CI_{95\%}$: (2.1 mg/dl, 12.2 mg/dl)) increase in the mean of total serum cholesterol.

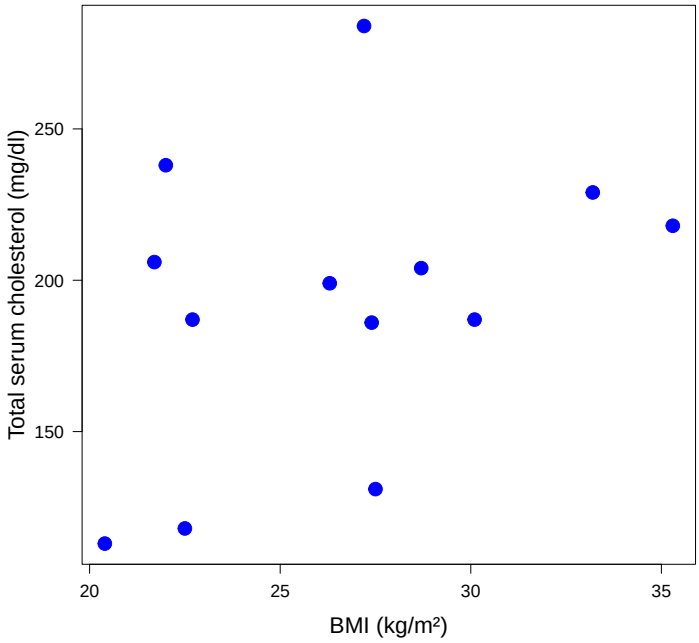


Figure 5.2: BMI and cholesterol in `nhanes2` data.

5.4.2 Simple mean imputation

Here, we create a new dataset which is the result of imputing the mean value of the variable. In the case of categorical variables, we decide, for instance, to impute the mode. Thus, we first create a function to do this simple mean imputation. To do this, we create before two auxiliary functions: `imputeMean()` and `imputeMode()`. Then, we create the function `imputeMeanMode()`, which internally calls `imputeMean()` or `imputeMode()`, depending on the type of variable (numerical or categorical, respectively):

```
imputeMean <- function(x) {
  xnew <- x
  xmean <- mean(x, na.rm = TRUE)
  xnew[is.na(x)] <- xmean
  return(xnew)
}
```

```
imputeMode <- function(x, seed = NULL) {
  xnew <- x
  freqs <- table(xnew)
  modepos <- which(freqs == max(freqs))
  # sampling if multimodal:
  l <- length(modepos)
  if (l > 1) {
    if (!is.null(seed))
      set.seed(seed)
    modepos <- sample(1:l, size = 1)
```

```
  }
  xmode <- levels(x)[modepos]
  xnew[is.na(x)] <- xmode
  return(xnew)
 }
```

```
imputeMeanMode <- function(x, seed = 666) {
  if (!anyNA(x))
    return(x)
  condNum <- inherits(x, c("numeric", "integer"))
  condFac <- inherits(x, "factor")
  condOther <- !condNum & !condFac
  if (condNum) xnew <- imputeMean(x = x)
  if (condFac) xnew <- imputeMode(x = x, seed = seed)
  if (condOther) xnew <- x
  return(xnew)
 }
```

Now we perform simple imputation **nhanes2** and store the imputed dataset in **nhanes2I**:

```
nhanes2I <- nhanes2
for (i in 1:dim(nhanes2I)[2])
 nhanes2I[, i] <- imputeMeanMode(nhanes2I[, i])
# for consistence, re-build bmig:
nhanes2I$bmig <- getBMIgroup(nhanes2I$bmi)
nhanes2I

##      age     bmi hyp   chl       bmig
## 1  20-39 26.5625  no 191.4 overweight
## 2  40-59 22.7000  no 187.0     normal
## 3  20-39 26.5625  no 187.0 overweight
## 4  60-99 26.5625  no 191.4 overweight
## 5  20-39 20.4000  no 113.0     normal
## 6  60-99 26.5625  no 184.0 overweight
## 7  20-39 22.5000  no 118.0     normal
## 8  20-39 30.1000  no 187.0      obese
## 9  40-59 22.0000  no 238.0     normal
## 10 40-59 26.5625  no 191.4 overweight
## 11 20-39 26.5625  no 191.4 overweight
## 12 40-59 26.5625  no 191.4 overweight
## 13 60-99 21.7000  no 206.0     normal
## 14 40-59 28.7000 yes 204.0 overweight
## 15 20-39 29.6000  no 191.4 overweight
## 16 20-39 26.5625  no 191.4 overweight
## 17 60-99 27.2000 yes 284.0 overweight
## 18 40-59 26.3000 yes 199.0 overweight
## 19 20-39 35.3000  no 218.0      obese
## 20 60-99 25.5000 yes 191.4 overweight
```

```
## 21 20-39 26.5625  no 191.4 overweight
## 22 20-39 33.2000  no 229.0      obese
## 23 20-39 27.5000  no 131.0 overweight
## 24 60-99 24.9000  no 191.4     normal
## 25 40-59 27.4000  no 186.0 overweight
```

We now fit our regression model to the imputed data `nhanes2I`, and save the result of interest in the object `effectI`:

```
effectI <- getBetaBMI(nhanes2I)
effectI


##   Estimate Std. Error   Pr(>|t|)      2.5 %     97.5 %
## 5.25426048 1.90317363 0.01205657 1.28430985 9.22421111
```

Interpretation: Under the fitted model using only simply imputed data, we estimate that each 1 kg/m^2 increase in BMI is associated with a 5.3 mg/dl ($CI_{95\%}$: (1.3 mg/dl, 9.2 mg/dl)) increase in the mean of total serum cholesterol.

5.4.3 Multiple imputation

In this case, we create a set of M imputed datasets using the multiple imputation technique. We can do this using the mice package which we have loaded previously.

5.4.3.1 Illustration

The `mice()` function in the mice package performs multiple imputation. Its main arguments are (see `?mice` for full details):

- `data`: A `data.frame` or a `matrix` containing the incomplete data. Missing values are coded as `NA`.
- `m`: The number of imputed datasets. Default is 5.
- `method`: The method of imputation used to impute each variable. It can be different for each variable. By default, it is internally set depending on the metric of the variable.
- `predictorMatrix`: A 0/1 matrix indicating which variables as used of predictors for the other variables. By default, all variables are predictors for the remaining variables (but not for the variable itself). I.e., a matrix with all elements equal to 1 except the diagonal, which is filled with zeros.
- `printFlag`: If `TRUE` (default), `mice()` will print history on console. Use `FALSE` for silent computation. `TRUE` is useful to see everything is going well; `FALSE` is useful to write the final analysis report.
- `seed`: An integer that is used as argument by the `set.seed()` function for offsetting the random number generator. Default is to leave the random number generator alone. In order to get reproducible results, you need to set `seed` (e.g., `seed = 666`).

First, we run the imputation model but with no iterations (i.e., no imputations are made), to get the default setting that R uses and then modify it to our convenience:

```
library(mice)
ini <- mice(nhanes2, maxit = 0)
ini ### note that age doesn't need to be imputed


## Class: mids
## Number of multiple imputations:  5
## Imputation methods:
##       age       bmi       hyp       chl      bmig
##        ""     "pmm"  "logreg"     "pmm" "polyreg"
## PredictorMatrix:
##      age bmi hyp chl bmig
## age    0   1   1   1    1
## bmi    1   0   1   1    1
## hyp    1   1   0   1    1
## chl    1   1   1   0    1
## bmig   1   1   1   1    0
```

Read the help of the `mice()` function for details on the available imputation methods.

Now, we need to make sure that, in the imputation process, data obey the relationship between numerical and categorical versions of BMI, i.e., `bmig = getBMIgroup(bmi)`. In addition, `bmig` cannot be a predictor when imputing `bmi`. I.e., `bmig` doesn't play any role in the imputation process for `bmi`. Instead, we set `mice()` so that it imputes `bmi` and then calculate the corresponding value of `bmig`. This type of restriction on the relationship between predictor variables and imputed variables during the imputation process is called *passive imputation*. Passive imputation can be set by customizing arguments `predictorMatrix` and `method` in the `mice()` function as follows:

```
### Get the imputation method and modify:
meth <- ini$method
### passive imputation for bmi:
meth["bmig"] <- "~getBMIgroup(bmi)"
### get the current predictorMatrix and modify in order "bmig" is not a predictor
### of "bmi":
pred <- ini$predictorMatrix
pred["bmi", "bmig"] <- 0
```

Now, we perform the multiple imputation process using, for instance, $M = 6$ imputations:

```
# Number of imputations:
M <- 6

# Multiple imputation (set printFlag = TRUE to see the progress):
imp <- mice(nhanes2,
            m = M,
            method = meth,
            predictorMatrix = pred,
```

```
            printFlag = FALSE,
            seed = 666)
```

Some results:

- Imputed values for each variable (columns correspond to imputations; rows correspond to individuals):

```
imp$imp

## $age
## [1] 1 2 3 4 5 6
## <0 rows> (o 0- extensión row.names)
##
## $bmi
##       1    2    3    4    5    6
## 1  26.3 29.6 29.6 27.2 29.6 30.1
## 3  29.6 26.3 35.3 27.2 33.2 30.1
## 4  21.7 25.5 27.5 22.7 25.5 20.4
## 6  25.5 27.4 25.5 22.7 27.4 27.4
## 10 22.5 27.5 28.7 27.5 20.4 22.0
## 11 30.1 29.6 29.6 35.3 29.6 30.1
## 12 22.7 26.3 27.2 27.4 28.7 27.5
## 16 27.2 26.3 33.2 25.5 29.6 29.6
## 21 35.3 30.1 33.2 27.2 27.2 30.1
##
## $hyp
##      1   2   3   4   5   6
## 1  yes yes yes  no yes yes
## 4   no yes  no  no  no yes
## 6   no yes  no  no yes yes
## 10 yes  no yes yes yes yes
## 11 yes  no yes  no yes  no
## 12 yes  no  no  no  no yes
## 16 yes  no yes  no yes  no
## 21  no yes yes  no yes  no
##
## $chl
##      1   2   3   4   5   6
## 1  204 218 187 187 218 186
## 4  199 206 218 187 229 186
## 10 204 206 284 204 204 238
## 11 229 186 199 218 206 187
## 12 184 238 218 184 187 204
## 15 187 131 187 187 238 187
## 16 218 113 186 238 218 187
## 20 184 206 206 218 229 218
## 21 113 218 186 187 218 187
## 24 131 206 184 199 229 184
```

```
##
## $bmig
##              1          2          3          4          5          6
## 1   overweight overweight overweight overweight overweight      obese
## 3   overweight overweight      obese overweight      obese      obese
## 4       normal overweight overweight     normal overweight     normal
## 6   overweight overweight overweight     normal overweight overweight
## 10      normal overweight overweight overweight     normal     normal
## 11       obese overweight overweight      obese overweight      obese
## 12      normal overweight overweight overweight overweight overweight
## 16 overweight overweight      obese overweight overweight overweight
## 21       obese      obese      obese overweight overweight      obese
```

- First imputed dataset:

```
complete(data = imp)
```

```
##      age  bmi hyp chl       bmig
## 1  20-39 26.3 yes 204 overweight
## 2  40-59 22.7  no 187     normal
## 3  20-39 29.6  no 187 overweight
## 4  60-99 21.7  no 199     normal
## 5  20-39 20.4  no 113     normal
## 6  60-99 25.5  no 184 overweight
## 7  20-39 22.5  no 118     normal
## 8  20-39 30.1  no 187      obese
## 9  40-59 22.0  no 238     normal
## 10 40-59 22.5 yes 204     normal
## 11 20-39 30.1 yes 229      obese
## 12 40-59 22.7 yes 184     normal
## 13 60-99 21.7  no 206     normal
## 14 40-59 28.7 yes 204 overweight
## 15 20-39 29.6  no 187 overweight
## 16 20-39 27.2 yes 218 overweight
## 17 60-99 27.2 yes 284 overweight
## 18 40-59 26.3 yes 199 overweight
## 19 20-39 35.3  no 218      obese
## 20 60-99 25.5 yes 184 overweight
## 21 20-39 35.3  no 113      obese
## 22 20-39 33.2  no 229      obese
## 23 20-39 27.5  no 131 overweight
## 24 60-99 24.9  no 131     normal
## 25 40-59 27.4  no 186 overweight
```

- Second imputed dataset:

```
complete(data = imp, action = 2)
```

```
##      age  bmi hyp chl       bmig
```

```
## 1  20-39 29.6 yes 218 overweight
## 2  40-59 22.7  no 187     normal
## 3  20-39 26.3  no 187 overweight
## 4  60-99 25.5 yes 206 overweight
## 5  20-39 20.4  no 113     normal
## 6  60-99 27.4 yes 184 overweight
## 7  20-39 22.5  no 118     normal
## 8  20-39 30.1  no 187      obese
## 9  40-59 22.0  no 238     normal
## 10 40-59 27.5  no 206 overweight
## 11 20-39 29.6  no 186 overweight
## 12 40-59 26.3  no 238 overweight
## 13 60-99 21.7  no 206     normal
## 14 40-59 28.7 yes 204 overweight
## 15 20-39 29.6  no 131 overweight
## 16 20-39 26.3  no 113 overweight
## 17 60-99 27.2 yes 284 overweight
## 18 40-59 26.3 yes 199 overweight
## 19 20-39 35.3  no 218      obese
## 20 60-99 25.5 yes 206 overweight
## 21 20-39 30.1 yes 218      obese
## 22 20-39 33.2  no 229      obese
## 23 20-39 27.5  no 131 overweight
## 24 60-99 24.9  no 206     normal
## 25 40-59 27.4  no 186 overweight
```

Now, we fit the regression model of interest to each of the 6 imputed dataset, and the pool the results as follows:

```
### 1) Fit the model to each of the imputed datasets:

modimp <- with(data = imp, exp = lm(chl ~ bmi + hyp + age))
modimp


## call :
## with.mids(data = imp, expr = lm(chl ~ bmi + hyp + age))
##
## call1 :
## mice(data = nhanes2, m = M, method = meth, predictorMatrix = pred,
##     printFlag = FALSE, seed = 666)
##
## nmis :
## [1]  0  9  8 10  9
##
## analyses :
## [[1]]
##
## Call:
## lm(formula = chl ~ bmi + hyp + age)
```

```
##
## Coefficients:
## (Intercept)          bmi       hypyes     age40-59     age60-99
##      72.642        3.357       32.313       26.539       32.610
##
##
## [[2]]
##
## Call:
## lm(formula = chl ~ bmi + hyp + age)
##
## Coefficients:
## (Intercept)          bmi       hypyes     age40-59     age60-99
##      -9.208        6.297        7.705       52.563       59.674
##
##
## [[3]]
##
## Call:
## lm(formula = chl ~ bmi + hyp + age)
##
## Coefficients:
## (Intercept)          bmi       hypyes     age40-59     age60-99
##      0.9959       5.7417      12.9746      59.9098      62.6017
##
##
## [[4]]
##
## Call:
## lm(formula = chl ~ bmi + hyp + age)
##
## Coefficients:
## (Intercept)          bmi       hypyes     age40-59     age60-99
##      29.633        5.409       11.039       25.292       49.245
##
##
## [[5]]
##
## Call:
## lm(formula = chl ~ bmi + hyp + age)
##
## Coefficients:
## (Intercept)          bmi       hypyes     age40-59     age60-99
##      42.848        4.881       17.299       27.590       51.520
##
##
## [[6]]
##
## Call:
## lm(formula = chl ~ bmi + hyp + age)
```

```
##
## Coefficients:
## (Intercept)          bmi       hypyes     age40-59     age60-99
##      13.927        5.583        3.476       51.224       57.201

### 2) Pool of fitted models:

modpool <- pool(modimp)
modpool

## Class: mipo    m = 6
##          term m  estimate        ubar         b           t dfcom        df
## 1 (Intercept) 6 25.139724 3148.284335 895.73554 4193.309129    20 11.715000
## 2         bmi 6  5.211555    3.725564   1.03667    4.935012    20 11.827056
## 3      hypyes 6 14.134515  234.666205 101.26869  352.813006    20  9.545588
## 4    age40-59 6 40.519536  307.672016 246.02321  594.699095    20  6.560043
## 5    age60-99 6 52.141875  359.356199 116.39988  495.156064    20 11.049767
##         riv    lambda       fmi
## 1 0.3319347 0.2492124 0.3512563
## 2 0.3246350 0.2450751 0.3469058
## 3 0.5034675 0.3348709 0.4409048
## 4 0.9328995 0.4826425 0.5908758
## 5 0.3778977 0.2742567 0.3775671

### 3) Get pooled estimates and confidence intervals:

coeffsMI <- summary(modpool, conf.int = TRUE, conf.level = 0.95)
coeffsMI

##          term  estimate std.error statistic        df    p.value        2.5 %
## 1 (Intercept) 25.139724 64.755765 0.3882237 11.715000 0.70481294 -116.3325973
## 2         bmi  5.211555  2.221489 2.3459741 11.827056 0.03726716    0.3634853
## 3      hypyes 14.134515 18.783317 0.7525036  9.545588 0.46989789  -27.9889606
## 4    age40-59 40.519536 24.386453 1.6615592  6.560043 0.14342572  -17.9385858
## 5    age60-99 52.141875 22.252102 2.3432336 11.049767 0.03885269    3.1922268
##      97.5 %     conf.low conf.high
## 1 166.61205 -116.3325973 166.61205
## 2  10.05962    0.3634853  10.05962
## 3  56.25799  -27.9889606  56.25799
## 4  98.97766  -17.9385858  98.97766
## 5 101.09152    3.1922268 101.09152

### 4) Get the association of interest:
effectMI <- coeffsMI[coeffsMI$term == "bmi",
                     c("estimate", "std.error", "p.value", "2.5 %", "97.5 %")]
names(effectMI) <- names(effectCC)
effectMI

##   Estimate Std. Error   Pr(>|t|)     2.5 %   97.5 %
## 2 5.211555   2.221489 0.03726716 0.3634853 10.05962
```

5.4.3.2 Comparison of results

Finally we can compare results otained with the different approaches:

```
results <- rbind(effectCC, effectI, effectMI)
rownames(results) <- c("complete cases",
                       "simple mean imputation",
                       "multiple imputation")
results
```

```
##                          Estimate Std. Error   Pr(>|t|)     2.5 %    97.5 %
## complete cases           7.160119   2.200899 0.01164394 2.0848377 12.235400
## simple mean imputation 5.254260   1.903174 0.01205657 1.2843099  9.224211
## multiple imputation    5.211555   2.221489 0.03726716 0.3634853 10.059625
```

Interpret results above regarding the impact of missingness on bias and precision of the estimation of the β coefficient of interest.

Chapter 6

Multiple imputation for clustering analysis (solved) (SHS/CDA)

6.1 Introduction

We will work with the object `minhanes` in the miclust package. `minhanes` is a *list* with 101 datasets (of class *data.frame*). The first dataset contains `nhanes` data from mice package. The remaining 100 datasets were obtained by applying the multiple imputation function `mice()` in the mice package. Here, I illustrate how to perform k-means clustering with variable selection while deciding the optimal number of cluster, in presence of missing data. To do that, we will integrate the such a clustering procedure in the aforementioned 100 imputed data sets.

6.2 Data

First, we load and explore `minhanes` data:

```
library(miclust)

data(minhanes)
class(minhanes)

## [1] "list"

length(minhanes)

## [1] 101

### raw data in the first element of "minhanes":
nhanes <- minhanes[[1]]
dim(nhanes)

## [1] 25  4

head(nhanes)

##   age  bmi hyp chl
```

```
## 1   1   NA  NA  NA
## 2   2 22.7   1 187
## 3   1   NA   1 187
## 4   3   NA  NA  NA
## 5   1 20.4   1 113
## 6   3   NA  NA 184
```

```
anyNA(nhanes)
```

```
## [1] TRUE
```

```
summary(nhanes)
```

```
##       age            bmi             hyp             chl
##  Min.   :1.00   Min.   :20.40   Min.   :1.000   Min.   :113.0
##  1st Qu.:1.00   1st Qu.:22.65   1st Qu.:1.000   1st Qu.:185.0
##  Median :2.00   Median :26.75   Median :1.000   Median :187.0
##  Mean   :1.76   Mean   :26.56   Mean   :1.235   Mean   :191.4
##  3rd Qu.:2.00   3rd Qu.:28.93   3rd Qu.:1.000   3rd Qu.:212.0
##  Max.   :3.00   Max.   :35.30   Max.   :2.000   Max.   :284.0
##                 NAs    :9       NAs    :8       NAs    :10
```

```
### first imputed data:
nhanes_imp1 <- minhanes[[2]]
### first imputed data set:
dim(nhanes_imp1)
```

```
## [1] 25  4
```

```
head(nhanes_imp1)
```

```
##   age  bmi hyp chl
## 1   1 35.3   1 186
## 2   2 22.7   1 187
## 3   1 28.7   1 187
## 4   3 25.5   2 218
## 5   1 20.4   1 113
## 6   3 27.5   2 184
```

```
anyNA(nhanes_imp1)
```

```
## [1] FALSE
```

```
summary(nhanes_imp1)
```

```
##       age            bmi             hyp             chl
##  Min.   :1.00   Min.   :20.40   Min.   :1.00   Min.   :113.0
```

```
##  1st Qu.:1.00   1st Qu.:22.70   1st Qu.:1.00   1st Qu.:186.0
##  Median :2.00   Median :27.40   Median :1.00   Median :187.0
##  Mean   :1.76   Mean   :26.74   Mean   :1.24   Mean   :191.7
##  3rd Qu.:2.00   3rd Qu.:28.70   3rd Qu.:1.00   3rd Qu.:206.0
##  Max.   :3.00   Max.   :35.30   Max.   :2.00   Max.   :284.0
```

The raw (incomplete) data set, defined above as `nhanes`, is stored as the first element of the *list* `minhanes`. The number of complete cases was 13 (52.0%). The percentage of missed values per variable is shown in Table 6.1. The `md.pattern()` function in the mice package provides further information on missingness, both numerical and graphical (see Figure 6.1):

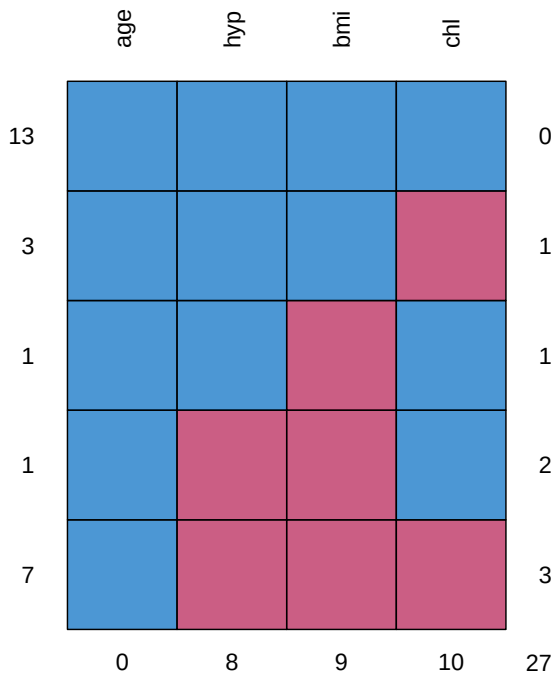


Figure 6.1: Missingness pattern in `nhanes` data (obtained with the `mice::md.pattern()` function). Blue means observed while maroon means missing. Numbers in the left column represent the number of observations with the corresponding row pattern (e.g., 13 complete cases, or 3 individuals missing only `chl`). Numbers in the right column represent the number of missing values in the corresponding row pattern. Numbers in the bottom row represent the number of individuals with missing value in the corresponding column variable (e.g., `age` is completely observed, or `hyp` value is missed for 8 individuals).

```
##    age hyp bmi chl
## 13   1   1   1   1  0
## 3    1   1   1   0  1
## 1    1   1   0   1  1
## 1    1   0   0   1  2
## 7    1   0   0   0  3
##      0   8   9  10 27
```

("1" means observed; "0" means missing)

✎ Complete the following paragraph by filling in the blanks with the corresponding results: "The subset used here included ____ participants and ____ variables. The range of missing data in the different variables went from ____% to ____% with ____% of the variables having less than 5% of the data missing. Only ____% of the participants had complete data on all variables. Overall, ____% of the information was missing; that is, ____% of the total of ____ cells (____ × ____ = ____) had missing data."

Variable	Missed values (%)
chl	40.0
bmi	36.0
hyp	32.0

Table 6.1: Percentage of missed values per variable in `nhanes` data set.

A summary of the variables in `obes` is shown in Table 6.2.[1]

	N	Missing	Mean	SD	Min	Q1	Median	Q3	Max
age	25	0	1.76	0.83	1.00	1.00	2.00	2.00	3.00
bmi	16	9	26.56	4.22	20.40	22.60	26.75	29.15	35.30
hyp	17	8	1.24	0.44	1.00	1.00	1.00	1.00	2.00
chl	15	10	191.40	45.22	113.00	185.00	187.00	212.00	284.00

Table 6.2: Summary of numeric variables in `nhanes` data set.

6.3 k-means clustering with variable selection in the presence of missing data

Now, we perform a k-means clustering in the `nhanes` data set taking into account that there are missing data in that data set. To do that, we will use the $M = 100$ imputed datasets stored in the *list* `minhanes`.

6.3.1 Data preparation for miclust package

We prepare the data with the `miclust::getdata()` function (see `?getdata` for details):

```
### prepare data for miclust:
library(miclust)
minhanes1 <- miclust::getdata(data = minhanes)
class(minhanes1)

## [1] "list"   "midata"

length(minhanes1)

## [1] 2
```

[1] Table 6.2 has been obtained using `papeR::summarize()` and then `xtable::xtable()`.

```
names(minhanes1)

## [1] "rawdata" "impdata"

### exploration of the raw data:
head(minhanes1$rawdata)

##   age  bmi hyp chl
## 1   1   NA  NA  NA
## 2   2 22.7   1 187
## 3   1   NA   1 187
## 4   3   NA  NA  NA
## 5   1 20.4   1 113
## 6   3   NA  NA 184

summary(minhanes1$rawdata)

##       age             bmi             hyp             chl
##  Min.   :1.00   Min.   :20.40   Min.   :1.000   Min.   :113.0
##  1st Qu.:1.00   1st Qu.:22.65   1st Qu.:1.000   1st Qu.:185.0
##  Median :2.00   Median :26.75   Median :1.000   Median :187.0
##  Mean   :1.76   Mean   :26.56   Mean   :1.235   Mean   :191.4
##  3rd Qu.:2.00   3rd Qu.:28.93   3rd Qu.:1.000   3rd Qu.:212.0
##  Max.   :3.00   Max.   :35.30   Max.   :2.000   Max.   :284.0
##                 NAs    :9       NAs    :8       NAs    :10

### exploration of the first imputed data set after transformations
### required to apply the clustering algorithm have been applied by
### the function ""getdata":
head(minhanes1$impdata[[1]])

##          age        bmi   hyp        chl
## 1 -0.9149325  2.0012331 -0.24 -0.1489984
## 2  0.2889260 -0.9445072 -0.24 -0.1227663
## 3 -0.9149325  0.4582263 -0.24 -0.1227663
## 4  1.4927846 -0.2898982  0.76  0.6904293
## 5 -0.9149325 -1.4822217 -0.24 -2.0639429
## 6  1.4927846  0.1776796  0.76 -0.2014626

summary(minhanes1$impdata[[1]])

##       age               bmi               hyp              chl
##  Min.   :-0.9149   Min.   :-1.4822   Min.   :-0.24   Min.   :-2.0639
##  1st Qu.:-0.9149   1st Qu.:-0.9445   1st Qu.:-0.24   1st Qu.:-0.1490
##  Median : 0.2889   Median : 0.1543   Median :-0.24   Median :-0.1228
##  Mean   : 0.0000   Mean   : 0.0000   Mean   : 0.00   Mean   : 0.0000
##  3rd Qu.: 0.2889   3rd Qu.: 0.4582   3rd Qu.:-0.24   3rd Qu.: 0.3756
##  Max.   : 1.4928   Max.   : 2.0012   Max.   : 0.76   Max.   : 2.4217
```

✎ As you can see in the output above, variables in the imputed data sets have been transformed. Read the help of `miclust::getdata()` to know what transformations have been done.

6.3.2 Clustering procedure

Now we apply the `miclust()` function. Suppose researchers have clinical reasons to suspect that individuals could be clustered into two or three groups, so they are interested in performing a cluster analysis considering both possibilities and deciding between one of the two possible groupings (i.e., between 2 or 3 clusters). In addition, they are interested in explore which variables are most important in determining the clustering. Hence, we could apply the `miclust()` function with the following setting:

```
nhanesmiclust <- miclust(data = minhanes1,
                         search = "backward",
                         ks = 2:3,
                         distance = "manhattan",
                         centpos = "means",
                         initcl = "hc",
                         verbose = TRUE,
                         seed = 666)
```

```
## . . . . imp   5 . . . . imp  10 . . . . imp  15 . . . . imp  20
## . . . . imp  25 . . . . imp  30 . . . . imp  35 . . . . imp  40
## . . . . imp  45 . . . . imp  50 . . . . imp  55 . . . . imp  60
## . . . . imp  65 . . . . imp  70 . . . . imp  75 . . . . imp  80
## . . . . imp  85 . . . . imp  90 . . . . imp  95 . . . . imp 100
##
##  Analysis done.
```

6.3.3 Clustering results

As we can see above, the clustering results have been stored in the object `nhanesmiclust`. By printing the object, we get a preliminary summary of results. Specifically, for each analysed number of clusters, k (in this case, $k = 2$ and $k = 3$), the output provides: 1) The percentage of imputed data sets for which that value of k was selected as the optimal number of clusters (compared to the other values of k), and 2) the median number of variables considered as the most important to determine the clustering:

```
### print results:
nhanesmiclust
```

```
##
##    Results using 100 imputations:
## -----------------------------------------------
##
##                                        k=2   k=3
## Frequency of selection (%)           56.00 44.00
## Median number of selected variables   1.0   2.0
```

We can also obtain a graphical summary of results (Figure 6.2):

```
### plot results:
plot(nhanesmiclust,
     col.nclfreq = "blue",
     col.critcf = "orange",
     col.nvarfreq = "forestgreen")
```

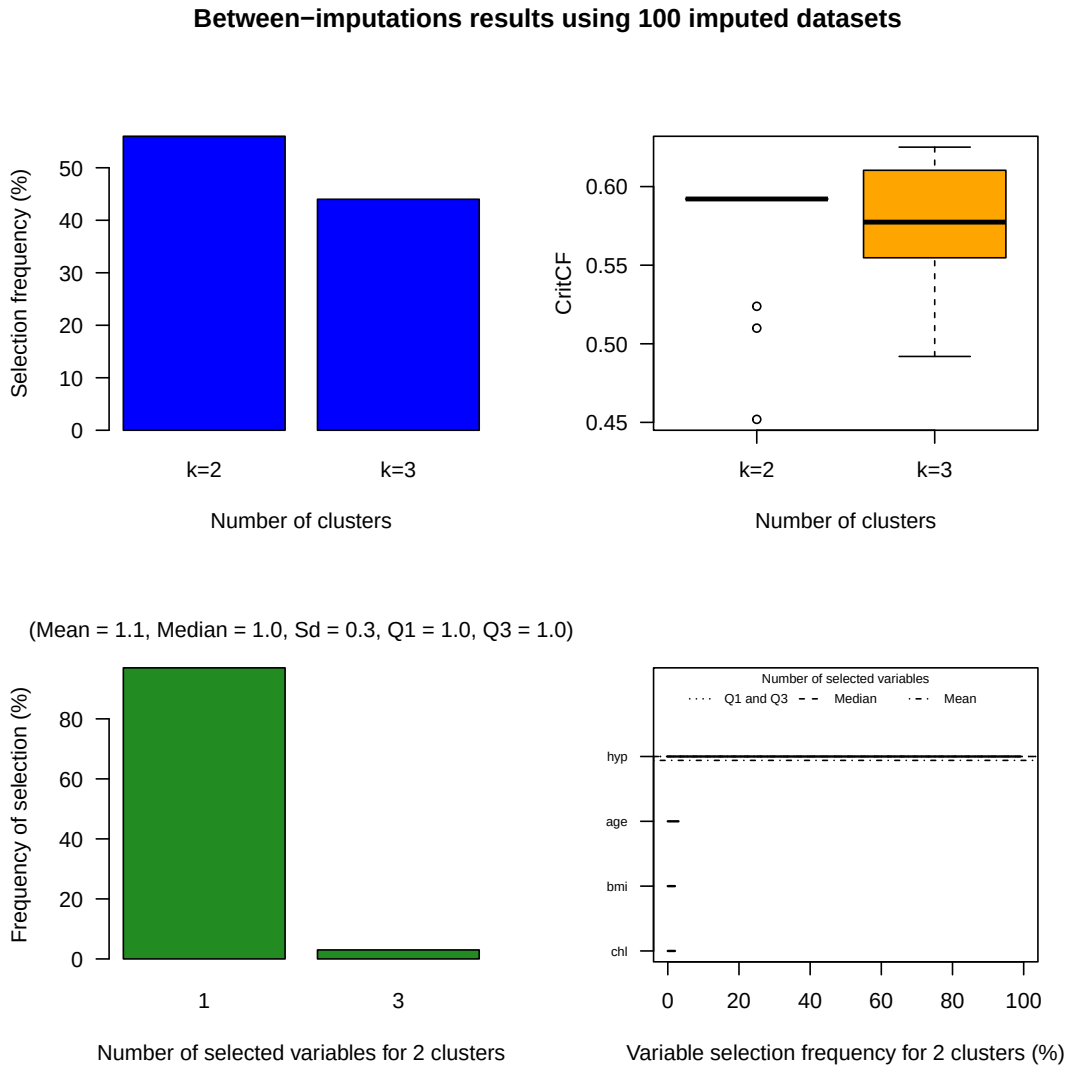


Figure 6.2: Graphical summary of the results of the clustering of the 100 imputed data sets in `minhanes` data. Customized colors are used.

Figure 6.2 shows the output of the method `plot()` for the following customized options:

- All available metrics are shown (`metric = "all"`, which is the default),
- A customized color for each metric, except for the variable selection frequency (`col.varsel`), for which the default `"black"` is used.

Read the help page (`?plot.miclust`) to learn how to choose which metrics are shown and how to use customized colors.

A more detailed summary of results can be obtained with the `summary()` method (by default, k is set to the optimal value found by `miclust()`):

```
nhanesmiclust_summary <- summary(nhanesmiclust, quantilevars = 0.5)
nhanesmiclust_summary


##
## Results using:
##     100 imputed datasets for the cluster analysis
##     100 imputed datasets for the descriptive summary
##     2 as the final number of clusters
## ------------------------------------------------------------
##
## Presence of the variables in the subset of selected variables:
##   Variable Presence(%)
## 1      hyp          99
## 2      age           3
## 3      bmi           2
## 4      chl           2
##
## Selected variables:
## [1] "hyp"
##
## Cohen's kappa between-imputations distribution (99 comparisons):
##  2.5%   25%   50%  mean   75% 97.5%
##  0.48  0.65  0.78  0.75  0.88  1.00
##
## Between-imputation clusters size distribution (100 imputations):
##           size min. 2.5% 25% 50% mean 75% 97.5% max.
## cluster 1    7    4    4   6   7  6.5   7     9   10
## cluster 2   18   15   16  18  18 18.5  19    21   21
##
## Probability of assignment to the cluster distribution (100 imputations):
##                          min   Q1 Q2 Q3 max
## Assigned to cluster 1 0.51 0.54  1  1   1
## Assigned to cluster 2 0.52 0.93  1  1   1
##
## Within-cluster summary (100 imputations) [%miss.;mean;sd]:
##     %miss.            cl.1            cl.2
## hyp     32 42.86;2.00;0.00 27.78;1.00;0.00
```

✎ In the input above, what does `quantilevars = 0.5` mean?

A graphical representation of the selected variables (Figure 6.3) can be obtained as follows:

```
### selected variables
y <- getvariablesfrequency(nhanesmiclust)
y


## $percfreq
```

```
## [1] 99  3  2  2
##
## $varnames
## [1] "hyp" "age" "bmi" "chl"
```

```
# plot skeleton:
par(las = 1)
plot(y$percfreq, type = "n", main = "",
     xlab = "Variable", ylab = "Percentage of times selected",
     xlim = 0.5 + c(0, length(y$varnames)),
     lwd = 5, col = "blue",
     xaxt = "n", yaxt = "n")  # for customized axes
# customized axes:
axis(1, at = 1:length(y$varnames), labels = y$varnames, las = 2)
aty <- 10 * (0:10)
axis(2, at = aty)
# background grid:
abline(h = aty, lty = 2, col = "gray")
# data bars:
lines(y$percfreq, type = "h", lwd = 5, col = "blue")
```

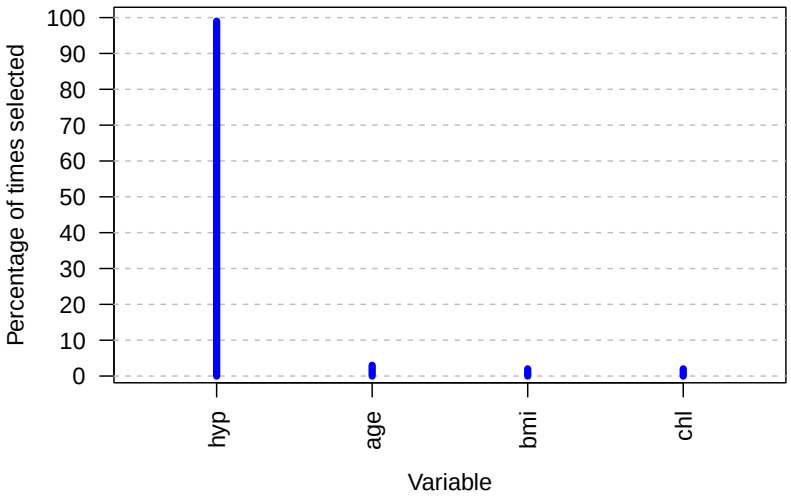


Figure 6.3: Graphical summary of variable selection for the results of the clustering of the 100 imputed data sets in `minhanes` data. Results for $k = 2$.

✎ Write a summary of the results about:

- The final number of clusters and its uncertainty
- The sizes of the clusters and their uncertainty
- The subset of selected variables and their uncertainty
- The probability with which individuals are assigned to the clusters

Chapter 7

Measures of the disease

7.1 Prevalence (SHS/CDA)

7.1.1 Estimating prevalences

We are interested in estimating the prevalence of asthma, π, in a population of interest. Specifically, our aim is to get both a point estimate and a 95% confidence interval. To do that, we select a random sample of size n from the population.

1. Explain what each of the lines of the following R script does.

```r
myBinomApprox <- function(x, n, conf.level = 0.95) {
  z <- qnorm((1 + conf.level) / 2)
  p <- x / n
  error <- z * sqrt(p * (1 - p) / n)
  lower <- p - error
  upper <- p + error
  res <- data.frame(x, n, proportion = p, lower, upper, conf.level)
  return(res)
}
```

2. Using several values for values for `x`, `n` and `conf.level`, check whether results of the functions `myBinomApprox()` and `binom.approx()` (package epitools) are identical.

3. Briefly explain what the function `binom.exact()` in the package epitools does.

4. Briefly explain what the R script shown in Appendix A does, and use it to explore the impact of both the sample size and the population prevalence π on the difference between the exact and the approximate confidence intervals.

7.1.2 Comparing prevalences

Suppose we consider, in a given population C, a binary health indicator D and a binary exposure E. We are interested in deciding about the two following hypothesis tests:

- Test 1:

$$\left.\begin{array}{ll} H_0: & P(D) = \pi_0 \\ H_1: & P(D) \neq \pi_0 \end{array}\right\}$$

- Test 2:

$$\left.\begin{array}{ll} H_0: & P(D \mid E) = P(D \mid \bar{E}) \\ H_1: & P(D \mid E) \neq P(D \mid \bar{E}) \end{array}\right\}$$

To do that, we have selected a random sample from C whose results are:

	Disease		
Exposure	No	Yes	**Total**
No	19	5	24
Yes	8	4	12
Total	27	9	36

1. What is testing each of the two tests?
2. Briefly explore what the functions `prop.test()` and `binom.test()` do.
3. Solve Test 1, for $\pi_0 = 0.14$, using both previous functions and interpret the results.
4. Solve Test 2 using the function `prop.test()` and interpret the results.
5. Explain what is the study design and how it affects results in questions 3 and 4.

7.1.3 Exact hypothesis test for a prevalence

1. We want to decide whether the prevalence of a certain chronic disease is equal to 0.3 or not. To do that, we set the significance level to $\alpha = 0.05$ and take a random sample of 11 individuals, resulting in only one of them having the disease. Apply an exact test (i.e., a hypothesis test based on the true probability distribution of the data under the null hypothesis), calculate the p-value and interpret the result. Solve the exercise by hand (i.e., without any computer) and then compare the result with the one obtained with the R function `binom.test()`.
2. In a given factory, the occupational safety regulations indicate that a worker has a safe working exposure level to a given pollutant (X) if it is not higher than 2.95 units. In this factory, previous studies have estimated that 95% of workers have an acceptable exposure level. A new security protocol has been implemented in order to increase the percentage of workers under safe levels of the pollutant. Some months after the implementation of the new protocol, we are interested in know if the percentage of workers with an acceptable exposure level has increased or if it is still the same (i.e., 95%). To make a decision, we randomly select a sample of 20 workers resulting in the following data:

Worker	1	2	3	4	5	6	7	8	9	10
$\boldsymbol{X}$	2.81	2.40	2.79	2.46	2.80	2.48	2.59	2.64	2.54	2.11

Worker	11	12	13	14	15	16	17	18	19	20
$\boldsymbol{X}$	2.37	2.66	2.29	2.54	2.91	2.78	2.63	2.73	2.89	2.41

Perform an exact hypothesis test in order to decide whether if the percentage of save workers has been increased after the implementation of the new protocol and interpret the

results. Calculate the p-value and interpret the result. Solve the exercise by hand (i.e., without any computer) and then compare the result with the one obtained with the R function `binom.test()`.

7.2 Incidence rate (SHS)

Table 7.1 shows data from a hypothetical cohort study.

	Exposed		**Non exposed**	
Age	Cases	*Person-years*	Cases	*Person-years*
35–44	32	52407	2	18790
45–54	104	43248	12	10673
55–64	206	28612	28	5710
65–74	186	12663	28	2585
75–84	102	5317	31	1462

Table 7.1: Data after the follow-up in a hypothetical cohort study.

1. Compute the incidence rate for each exposure group and age group.
2. For individuals ranging between 45 and 54 years, compute a confidence interval for the incidence rate, using the normal distribution.
3. For individuals ranging between 65 and 74 years, test the hypothesis of equailty of incidence rates in both exposure groups.
4. Estimate the cummulative incidence for a 10-years period for an individual aged 35, irrespective of the exposure status.

Chapter 8

Analysis of the contingency table for crude association assessment (SHS/CDA)

1. In a study, two simple random samples of sizes 660 and 527 were selected from populations of people with high cholesterol and healthy cholesterol levels, respectively. There were 218 and 158 people who used to follow a high-fat diet, respectively. Setting a significance level $\alpha = 0.05$ and assuming an asymptotic approximation to the distribution of an appropriate measure of association, based on the data and calculations you can make **without any computer**, answer the following questions:

   (a) What is the appropriate measure of the crude association between cholesterol level and diet? Why?

   (b) Provide a point estimate and a 95% confidence interval and interpret the results.

   (c) Based on the results, what could you say about the association between cholesterol level and diet?

2. In a study to assess the crude relationship between cholesterol level (high or healthy) and diet (high-fat or low-fat), a simple random sample of 707 individuals was selected from the population of interest and, for each individual, the level of fat in the diet and the level of cholesterol were recorded. When analysing the sample, it was observed that there were 577 and 130 individuals with high and healthy cholesterol levels, respectively. The number of individuals with high-fat diet was 365 and 59 within the subgroups of individuals with high and healthy cholesterol levels, respectively. Setting a significance level $\alpha = 0.05$ and assuming asymptotic approximation to the distribution of an appropriate measure of association, based on the data and calculations you can make **without any computer**, answer the following questions:

   (a) What is the appropriate measure of the crude association between cholesterol level and diet? Why?

   (b) Provide a point estimate and a 95% confidence interval and interpret the results.

   (c) Based on the results, what could you say about the association between cholesterol level and diet?

3. In a study, two simple random samples of sizes 424 and 283 were selected from populations of people with high-fat diet and low-fat diet, respectively. All individuals had healthy cholesterol levels. After a 5-year follow-up, 365 individuals from the high-fat diet group and 212 individuals from the low-fat diet group showed high, non healthy cholesterol levels. Setting a significance level $\alpha = 0.05$ and assuming an asymptotic approximation to the distribution of an appropriate measure of association, based on the data and calculations you can make **without any computer**, answer the following questions:

(a) What is the appropriate measure of the crude association between cholesterol level and diet? Why?

(b) Provide a point estimate and a 95% confidence interval and interpret the results.

(c) Based on the results, what could you say about the association between cholesterol level and diet?

4. Solve the three previous exercises using `epiR::epi.2by2()`.

Chapter 9

Estimating OR in a matched case-control study (SHS)

9.1 Introduction

The matched case-control design can be used to control the confusion of C when estimating the OR of a disease D associated with an exposure E. For instance, in a case-control study on the effect of alcohol consumption (E) on oral cancer (D), cases and controls should be comparable with respect to smoking habits (C). A possible strategy is *pair matching* or *1:1 matching*: for each case, one control is chosen with the same level of C (e.g., both smokers or both non-smokers). This type of matching allows to control for C when assessing the association between D and E.

9.2 Review of (unmatched) case-control design

In the simple case-control design, individuals are sampled independently. We directly cross-tabulate their disease status (rows) with their exposure level (columns), as shown in Table 9.1, where n_{yx} denotes the number of individuals with disease status y ($y = 1$ for cases, and $y = 0$ for controls), and exposure level x ($x = 1$ for exposed, and $x = 0$ for unexposed).

	Exposed	
Disease	No	Yes
No (control)	n_{00}	n_{01}
Yes (case)	n_{10}	n_{11}

Table 9.1: Contingency table structure for an unmatched case-control study.

Under this study design, the OR is estimated as

$$\widehat{\text{OR}} = \frac{n_{11} \cdot n_{00}}{n_{01} \cdot n_{10}}.$$

An approximate confidence interval is calculated under a normal approximation on the logarithmic scale, which is then transformed back using the exponential function, resulting in the formula

$$\text{CI}_{1-\alpha}(\text{OR}) \approx \widehat{\text{OR}} \cdot \exp\left(\pm z_{1-\alpha/2}\sqrt{\widehat{\text{var}}(\log(\widehat{\text{OR}}))}\right),$$

where the variance of $\log(\widehat{\text{OR}})$ is the sum of the reciprocals of the four cells, historically known as Woolf's Formula[5]:

$$\widehat{\text{var}}(\log(\widehat{\text{OR}})) \approx \frac{1}{n_{11}} + \frac{1}{n_{10}} + \frac{1}{n_{01}} + \frac{1}{n_{00}}.$$

9.2.1 Illustrative example (unmatched)

We study the relationship between smoking and lung cancer in 100 cases and 100 controls. Results were: $n_{00} = 60$, $n_{01} = 40$, $n_{10} = 20$, and $n_{11} = 80$.

- Point Estimation:
$$\widehat{\text{OR}} = \frac{80 \cdot 60}{40 \cdot 20} = 6$$
- Variance of log(OR):
$$\widehat{\text{var}}(\log(\widehat{\text{OR}})) = \frac{1}{80} + \frac{1}{20} + \frac{1}{40} + \frac{1}{60} = \frac{5}{48}$$
- 95% confidence interval:
$$\text{CI}_{95\%}(\text{OR}) \approx 6 \cdot \exp\left(\pm 1.96\sqrt{5/48}\right) \approx (3.19, 11.29)$$

9.3 Matched case-control studies (1:1 matching)

9.3.1 Types of pairs

In a *pair matching* or *1:1 matching*, for each case, one control is chosen with the same level of C. The n pairs of cases (D) and controls ($\bar{D}$) can be classified according to exposure (E) status in the four types shown in Table 9.2.

Type 1	E	$\bar{E}$	**Total**
D	1	0	1
$\bar{D}$	1	0	1
Total	2	0	2

Type 3	E	$\bar{E}$	**Total**
D	0	1	1
$\bar{D}$	1	0	1
Total	1	1	2

Type 2	E	$\bar{E}$	**Total**
D	1	0	1
$\bar{D}$	0	1	1
Total	1	1	2

Type 4	E	$\bar{E}$	**Total**
D	0	1	1
$\bar{D}$	0	1	1
Total	0	2	2

Table 9.2: The four types of classification of the pairs in a 1:1 matching case-control design. Pairs in types 1 and 4 are concordant while pairs in types 2 and 3 are discordant.

9.3.2 Estimation of the OR

Focusing in Table 9.2, we can see that:

- *Concordant pairs:* In types 1 and 4, cases and controls agree with respect to E.
- *Discordant pairs:* In types 2 and 3, cases and controls disagree with respect to E.
- Hence, the discordant pairs in types 2 and 3 include the useful information about the direction of the association between E and D.

- Thus, pairs in types 2 and 3 are used to estimate the OR.

Let n_i, $i = 1, \ldots, 4$, be the number of pairs of type i and p_j the exposure probability for cases ($j = 1$) and controls ($j = 0$). Then, given a discordant pair, the probability that it is of type 2 (case is exposed) is [6]

$$\pi \overset{\text{def}}{=} \text{P(Exposed case} \mid \text{Discordant pair)} = \frac{p_1(1-p_0)}{p_1(1-p_0)+(1-p_1)p_0} = \frac{\text{OR}}{\text{OR}+1}. \tag{9.1}$$

✎ Prove equation (9.1) (solution in Appendix B).

As a consequence of equation (9.1),

$$\text{OR} = \frac{\pi}{1-\pi} \implies \widehat{\text{OR}} = \frac{\hat{\pi}}{1-\hat{\pi}},$$

and, since $\hat{\pi} = n_2/(n_2+n_3)$ and $1-\hat{\pi} = n_3/(n_2+n_3)$, we get

$$\widehat{\text{OR}}_{\text{matched}} = \frac{n_2}{n_3}. \tag{9.2}$$

Note that to compute $\widehat{\text{OR}}_{\text{matched}}$ in equation 9.2, only information on discordant pairs is used.

A confidence interval for the odds ratio based on the asymptotic normal distribution of its logarithm is

$$\text{CI}_{100(1-\alpha)\%}(\text{OR}_{\text{matched}}) = \widehat{\text{OR}}_{\text{matched}} \cdot \exp\left(\pm z_{1-\alpha/2} \cdot \sqrt{\widehat{\text{var}}(\log(\widehat{\text{OR}}_{\text{matched}}))}\right), \tag{9.3}$$

where the asymptotic variance is estimated as (Miettinen's Formula) [6,7]:

$$\widehat{\text{var}}(\log(\widehat{\text{OR}}_{\text{matched}})) \approx 1/n_2 + 1/n_3. \tag{9.4}$$

9.3.3 McNemar test

The McNemar test may be used to test H_0: OR = 1:

$$\chi^2 = \frac{(n_2-n_3)^2}{n_2+n_3} \overset{\text{H}_0}{\sim} \chi^2_1.$$

In R: `stats::mcnemar.test()` (only p-value) or `epibasix::mcNemar()` (both p-value and confidence interval).

9.3.4 Comments

- In the case of controls being easy to find and inexpensive to evaluate, statistical efficiency may be increased by 1:M or n:M matching. However, according to some authors [8], the gain beyond four or five controls per case is minimal.
- Overmatching bias occurs if C is associated with E but not with D. It causes underestimation of the association between E and D.

(a) Overall

	Control	
Case	E	$\bar{E}$
E	29	87
$\bar{E}$	28	82

(b) Females

	Control	
Case	E	$\bar{E}$
E	9	29
$\bar{E}$	19	33

(c) Males

	Control	
Case	E	$\bar{E}$
E	20	58
$\bar{E}$	9	49

Table 9.3: Data obtained in a 1:1 matching case-control study.

9.3.5 Exercise

A case-control study on disease D was conducted to investigate its possible association with exposure to E. Researcher A, knowing that this was a matched study, collected the data from the 226 pairs as shown in Table 9.3.

1. Researcher A created the function `ormatched(n1234, alpha = 0.05)`, which includes the two following arguments:

   - `n1234`: a numeric vector with values n_1, n_2, n_3 and n_4 in the matched contingency table. For instance, in Table 9.3(c), `n1234 = c(29, 87, 28, 82)`.
   - `alpha`: the significance level for the McNemar test (set the default to 0.05).

   The output of `ormatched()` is a *data.frame* with 3 rows and 6 columns:

   - Rows correspond to calculation methods: row 1 for results using formulas for the asymptotic approximation; row 2 for results using `epibasix::mcNemar()`; and row 3 for results using `stats::mcnemar.test()`.
   - Columns: column 1 indicates the calculation method; columns 2 to 6 correspond to: $\text{OR}_{\text{matched}}$, lower and upper bounds of the confidence interval, χ^2 statistic, and p-value of the McNemar test (H_0: $\text{OR} = 1$), respectively.

2. Create your own `ormatched()` function and apply it to data in Table 9.3. You should get the following results:

```
# overall:
ormatched(n1234 = n1234o, alpha = 0.05)

##                method OR_matched lower_95% upper_95%     chi2      p-value
## 1          Asymptotic   3.107143  2.029627  4.756705 30.26957 3.759794e-08
## 2   epibasix::mcNemar   3.107143  2.064979  5.223162 29.25217 6.354489e-08
## 3 stats::mcnemar.test         NA        NA        NA 29.25217 6.354489e-08

# female:
ormatched(n1234 = n1234f, alpha = 0.05)

##                method OR_matched lower_95% upper_95%     chi2   p-value
## 1          Asymptotic   1.526316 0.8558765  2.721935 2.083333 0.1489147
## 2   epibasix::mcNemar   1.526316 0.8362285  3.047401 1.687500 0.1939309
## 3 stats::mcnemar.test         NA        NA        NA 1.687500 0.1939309
```

```r
# male:
ormatched(n1234 = n1234m, alpha = 0.05)
```

```
##                method OR_matched lower_95% upper_95%     chi2      p-value
## 1          Asymptotic   6.444444  3.193238  13.00588 35.83582 2.146646e-09
## 2   epibasix::mcNemar   6.444444  3.475397  21.11760 34.38806 4.514846e-09
## 3 stats::mcnemar.test         NA        NA        NA 34.38806 4.514846e-09
```

(a) Interpret the previous results regarding the association between E and D for the overall sample as well as when analysing each sex.

(b) Could sex be modifying the association between E and D? Why?

(c) What results would you report to summarize the association between E and D?

3. Researcher B, on the other hand, is unaware of the pairing and has collected the data for the 452 individuals separately. What table would researcher B obtain? That is, fill in Table 9.4.

(a) Overall

	Exposure	
Disease	E	$\bar{E}$
D	?	?
$\bar{D}$	?	?

(b) Females

	Exposure	
Disease	E	$\bar{E}$
D	?	?
$\bar{D}$	?	?

(c) Males

	Exposure	
Disease	E	$\bar{E}$
D	?	?
$\bar{D}$	?	?

Table 9.4: Unpaired data structure for the 1:1 matching case-control study of Table 9.3.

(a) Based on data in Table 9.4, estimate OR (both point estimate and 95% confidence interval) for the overall sample as well as for each sex. Use `epiR::epi.2by2()`.

(b) Interpret the results in terms of the possible association between E and D, taking into account the possible role of sex.

(c) Compare results with those obtained by researcher A. What is the cause of the possible discrepancies?

(d) Which researcher performed a more appropriate analysis? Why?

9.4 Consequences of ignoring matching in the analysis: Confounding control

If an investigator conducts a matched case-control study to control for confounding variables (such as age or sex) but decides to "break" the pairs and analyse the crude pooled data, they will introduce significant methodological errors.

Impact on point estimation ($\widehat{\text{OR}}$) The point estimation changes. If matching is ignored in the analysis of a matched study, control of confounding due to matching variable(s) is lost. Hence, the estimated OR will be systematically biased.

Impact on the confidence interval The confidence interval also changes fundamentally because the asymptotic variances are computed under different mathematical assumptions:

- **Unmatched analysis:** It assumes that all individual observations are independent. It computes the variance using all four cells $(n_{11}, n_{10}, n_{01}, n_{00})$. It produces an artificially smaller variance, narrowing the CI incorrectly.
- **Matched analysis:** Recognizes that the observations are dependent (linked as pairs). The variance is derived strictly from the discordant pairs (n_2, n_3). It yields a realistic variance of and a valid, adjusted (for the confounders that define the pairing) confidence interval.

9.5 Alternative when matching by design is unfeasible

There are situations where controlling for confounding through strict matching during the design phase becomes logistically impossible or inefficient. This typically occurs when the confounding variables are continuous scales (such as numeric age, blood pressure, or laboratory biomarkers), making it hard to find an identical control match for every single case.

In these scenarios, instead of forcing a physical match during data collection, we can conduct an unmatched study and address confounding control during the data analysis phase using *logistic regression models*. This statistical approach allows us to estimate an adjusted OR that mathematically accounts for multiple continuous or categorical variables simultaneously, achieving the same confounding control without the sampling restrictions of physical matching. In addition, multivariable logistic regression models allows us to explore potential interaction effects.

Chapter 10

Estimating risks under different sampling designs* (SHS)

The aims of this exercise are:

- Estimate the risks of a binary disease, D, related to a binary exposure, E, under different sampling designs.
- Illustrate results with R.

Suppose we know that, in a given population, the exposure E is a risk factor for the disease D, both of them binary. Consider the following population parameters:

- $P(E) = \pi_E$
- $P(D \mid \bar{E}) = \pi_0$
- $P(D \mid E) = \pi_1 > \pi_0$

1. Write expressions for $\pi_D \stackrel{\text{def}}{=} P(D)$, $P(E \mid \bar{D})$ and $P(E \mid D)$, as a function of π_E, π_0 and π_1. (**Hint:** Use formulae for conditional probability and total probability.)
2. Suppose we perform the three following sampling designs in this population:
   - Sample 1: We randomly select n individuals and then we measure the E and D status in each one of them.
   - Sample 2: We randomly select $n_{\bar{E}}$ non exposed individuals and n_E exposed individuals, all of them disease free, such that $n_{\bar{E}} + n_E = n$. Then, after a long follow-up in relation with the latency of the disease, we measure the D status in each one of them.
   - Sample 3: We randomly select $n_{\bar{D}}$ controls and n_D cases, such that $n_{\bar{D}} + n_D = n$. Then, we ask each of them for the E status.

   For each one of the previous samples:

   (a) Identify the study design.

   (b) Create a 2×2 contingency table ($E \times D$) with the expected values of each cell, as a function of the parameters π_E, π_0, π_1, n_D, n_E and n.

   (c) According to results obtained, fill in the following table:

	Can we get an unbiased estimate for . . . ?					
Study design	$P(E)$	$P(D)$	$P(D \mid \bar{E})$	$P(D \mid E)$	$P(E \mid \bar{D})$	$P(E \mid D)$

3. Create an R function `estimateRisks()` like the following one:

```
estimateRisks <- function(n, piE, pi0, pi1, propE, propD, seed = NULL) {
  ...
}
```

Arguments of `estimateRisks()`:

- `n`: the sample size, n
- `piE`: the value of $\pi_E = P(E)$
- `pi0`: the value of $\pi_0 = P(D \mid \bar{E})$
- `pi1`: the value of $\pi_1 = P(D \mid E)$
- `propE`: the proportion of exposed in Sample 2, $\frac{n_E}{n}$
- `propD`: the proportion of cases in Sample 3, $\frac{n_D}{n}$
- `seed`: the seed for reproducibility of results. Default is `NULL` (and then the results can't be reproduced)

The function `estimateRisks()` should simulate the three samplings and then return a *list* containing the three following objects:

- `tables2x2`, the three 2×2 contingency tables (one for each study design) resulting from the sampling simulation.
- `estimates`, a summary table showing the comparison between the estimates and the population parameters.
- `percentBias`, a table showing the biases (in %) of the estimates.

The output of `estimateRisks()` should be as in the following example:

```
estimateRisks(n = 20000, piE = 1/5, pi0 = 1/25, pi1 = 1/10, propE = 1/2,
              propD = 1/2, seed = 4321)

## $tables2x2
## $tables2x2$Design1
##       non D    D Total
## non E 15419  658 16077
## E      3525  398  3923
## Total 18944 1056 20000
##
## $tables2x2$Design2
##       non D    D Total
```

```
## non E  9660  340 10000
## E      9006  994 10000
## Total 18666 1334 20000
##
## $tables2x2$Design3
##       non E    E Total
## non D  8068 1932 10000
## D      6112 3888 10000
## Total 14180 5820 20000
##
##
## $estimates
##                P(E)   P(D)  P(D|nonE)    P(D|E) P(E|nonD)    P(E|D)
## Design1    0.19615 0.0528 0.04092803 0.1014530 0.1860747 0.3768939
## Design2    0.50000 0.0667 0.03400000 0.0994000 0.4824815 0.7451274
## Design3    0.29100 0.5000 0.43102962 0.6680412 0.1932000 0.3888000
## Population 0.20000 0.0520 0.04000000 0.1000000 0.1898734 0.3846154
##
## $percentBias
##          P(E)  P(D) P(D|nonE) P(D|E) P(E|nonD) P(E|D)
## Design1  -1.9   1.5       2.3    1.5      -2.0   -2.0
## Design2 150.0  28.3     -15.0   -0.6     154.1   93.7
## Design3  45.5 861.5     977.6  568.0       1.8    1.1
```

(**Hint:** You can use the (incomplete) code in the Appendix C.)

4. Set the following values of the parameters:

   - $P(E) = 1/5$
   - $P(D \mid \bar{E}) = 1/25$
   - $P(D \mid E) = 1/10$
   - $n = 20000$

   Then, vary the values of `propE` and `propD` and check whether results are consistent with the table obtained in question (2c).

5. Write your conclusions in a report extending no more than 10 lines.

Chapter 11

The logistic regression model (CDA)

11.1 Data simulation

Consider the logistic regression model

$$\log\left(\frac{\pi}{1-\pi}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 E + \beta_4 X_1 \cdot E + \beta_5 X_2 \cdot E, \tag{11.1}$$

where

- $\pi \overset{\text{def}}{=} P(D = 1 \mid E, X_1, X_2)$,
- D is the binary response variable,
- E is the binary exposure of interest,
- $X_1 \sim \mathcal{N}(\mu_1, \sigma_1)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2)$ are independent normally distributed covariates, which are potentially associated to D.

1. Prove the following expressions:

$$\begin{aligned}
\beta_1 &= \log \mathrm{OR}(D \mid E = 0)_{\{\Delta X_1 = 1\}} \\
\beta_2 &= \log \mathrm{OR}(D \mid E = 0)_{\{\Delta X_2 = 1\}} \\
\beta_0 &= \log \mathrm{odds}(D \mid X_1 = \mu_1, X_2 = \mu_2, E = 0) - \beta_1 \mu_1 - \beta_2 \mu_2 \\
\beta_4 &= \log \mathrm{OR}(D \mid E = 1)_{\{\Delta X_1 = 1\}} - \beta_1 \\
\beta_5 &= \log \mathrm{OR}(D \mid E = 1)_{\{\Delta X_2 = 1\}} - \beta_2 \\
\beta_3 &= \log \mathrm{odds}(D \mid X_1 = \mu_1, X_2 = \mu_2, E = 1) - \beta_4 \mu_1 - \beta_5 \mu_2
\end{aligned}$$

2. Explore the code corresponding to the `simulateData()` function in order to understand how the previous expressions are used to simulate databases containing the variables E, X_1, X_2 and D, according to model 11.1.

```
##############################################################################
##############################################################################
##############################################################################
###
### simulateData: Function to simulate a database with a binary response
###               (D), two independent normal covariates (X1, X2), and a
###               binaryexposure (E).
###
```

```
##############################################################################
###
### Input:
###
### n:            sample size
### muX:          vector with the means of X1, X2
### sigmaX:       vector with the sd of X1, X2
### pE:           prevalence of E
### pDmuXE0:      P(D|X=E(X), E=0)
### pDmuXE1:      P(D|X=E(X), E=1)
### ORDeltaX1E0: OR(DeltaX1=1|E=0)
### ORDeltaX1E1: OR(DeltaX1=1|E=1)
### ORDeltaX2E0: OR(DeltaX2=1|E=0)
### ORDeltaX2E1: OR(DeltaX2=1|E=1)
### seed:         seed for reproducibility of results.
###
##############################################################################
###
### Output:
###
### a simulated database
###
##############################################################################
##############################################################################
##############################################################################

simulateData <- function(n,
                         muX,
                         sigmaX,
                         pE,
                         pDmuXE0,
                         pDmuXE1,
                         ORDeltaX1E0,
                         ORDeltaX1E1,
                         ORDeltaX2E0,
                         ORDeltaX2E1,
                         seed = NULL) {
  if (!is.null(seed))
   set.seed(seed)
  X1 <- rnorm(n, mean = muX[1], sd = sigmaX[1])
  X2 <- rnorm(n, mean = muX[2], sd = sigmaX[2])
  E <- rbinom(n = n, size = 1, prob = pE)
  b1 <- log(ORDeltaX1E0)
  b2 <- log(ORDeltaX2E0)
  b0 <- log(pDmuXE0 / (1 - pDmuXE0)) - b1 * muX[1] - b2 * muX[2]
  b4 <- log(ORDeltaX1E1) - b1
  b5 <- log(ORDeltaX2E1) - b2
  numerb3 <- (1 - pDmuXE0) * pDmuXE1
  denomb3 <- pDmuXE0 * (1 - pDmuXE1)
  b3 <- log(numerb3 / denomb3) - b4 * muX[1] - b5 * muX[2]
```

```
  L <- b0 + b1 * X1 + b2 * X2 + b3 * E + (b4 * X1 + b5 * X2) * E
  pD = 1 / (1 + exp(-L))
  D <- apply(X = as.matrix(pD),
             MARGIN = 1,
             FUN = function(x) rbinom(n = 1, size = 1, prob = x))
  sim <- data.frame(X1, X2, E, D)
  return(sim)
}
```

3. Explore the following chunk corresponding to the simulation of the datasets `dat0`, `dat1`, `dat2` and `dat3`. Check that the required constraints in the parameters of the model are those shown in each case.

```
##############################################################################
##############################################################################
###
### Simulated datasets:
###
##############################################################################
##############################################################################


##############################################################################
### Simulating a database in which D depends only on E. This is equivalent
### to (check it):
# b1 = 0 --> ORDeltaX1E0 = 1
# b2 = 0 --> ORDeltaX2E0 = 1
# b4 = 0 --> ORDeltaX1E1 = 1
# b5 = 0 --> ORDeltaX2E1 = 1
##############################################################################

dat0 <- simulateData(n = 5000,
                     muX = c(10, 20),
                     sigmaX = c(1, 3),
                     pE = 0.10,
                     pDmuXE0 = 0.05,
                     pDmuXE1 = 0.25,
                     ORDeltaX1E0 = 1,
                     ORDeltaX1E1 = 1,
                     ORDeltaX2E0 = 1,
                     ORDeltaX2E1 = 1,
                     seed = 666)

##############################################################################
### Simulating a database in which D depends only on both E and X1, with
### no interactions. This is equivalent to (check it):
# b2 = 0 --> ORDeltaX2E0 = 1
# b4 = 0 --> ORDeltaX1E1 = 1
# b5 = 0 --> ORDeltaX2E1 = 1
##############################################################################
```

```
dat1 <- simulateData(n = 5000,
                     muX = c(10, 20),
                     sigmaX = c(1, 3),
                     pE = 0.10,
                     pDmuXE0 = 0.05,
                     pDmuXE1 = 0.25,
                     ORDeltaX1E0 = 1.15,
                     ORDeltaX1E1 = 1,
                     ORDeltaX2E0 = 1,
                     ORDeltaX2E1 = 1,
                     seed = 666)

##############################################################################
### Simulating a database in which D depends only on both E and X1, and
### the interaction (E * X1). This is equivalent to (check it):
# b2 = 0 --> ORDeltaX2E0 = 1
# b5 = 0 --> ORDeltaX2E1 = 1
##############################################################################

dat2 <- simulateData(n = 5000,
                     muX = c(10, 20),
                     sigmaX = c(1, 3),
                     pE = 0.10,
                     pDmuXE0 = 0.05,
                     pDmuXE1 = 0.25,
                     ORDeltaX1E0 = 1.15,
                     ORDeltaX1E1 = 1.50,
                     ORDeltaX2E0 = 1,
                     ORDeltaX2E1 = 1)

##############################################################################
### Simulating a database in which D depends only on E, X1 and X2, with
### no interactions. This is equivalent to (check it):
# b4 = 0 --> ORDeltaX1E1 = 1
# b5 = 0 --> ORDeltaX2E1 = 1
##############################################################################

dat3 <- simulateData(n = 5000,
                     muX = c(10, 20),
                     sigmaX = c(1, 3),
                     pE = 0.10,
                     pDmuXE0 = 0.05,
                     pDmuXE1 = 0.25,
                     ORDeltaX1E0 = 1.15,
                     ORDeltaX1E1 = 1,
                     ORDeltaX2E0 = 1.20,
                     ORDeltaX2E1 = 1)
```

4. Execute the previous chunk to get `dat0`, `dat1`, `dat2` and `dat3`.

11.2 Fitting the logistic regression model

1. Explore the following chunk which is used to fit the logistic regression model. Find out how the interactions are set.

```
mod0 <- glm(D ~ E, data = dat0, family = binomial)
mod1 <- glm(D ~ E + X1, data = dat1, family = binomial)
mod2 <- glm(D ~ E * X1, data = dat2, family = binomial)
mod3 <- glm(D ~ E + X1 + X2, data = dat3, family = binomial)
```

2. Execute the previous chunk to get `mod0`, `mod1`, `mod2` and `mod3`. Then, explore the result of fitting these models using the `summary()` method:

```
summary(mod0)

##
## Call:
## glm(formula = D ~ E, family = binomial, data = dat0)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.96718    0.06882  -43.12   <2e-16 ***
## E            1.84830    0.12796   14.45   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 2463.4  on 4999  degrees of freedom
## Residual deviance: 2289.5  on 4998  degrees of freedom
## AIC: 2293.5
##
## Number of Fisher Scoring iterations: 5

summary(mod1)

##
## Call:
## glm(formula = D ~ E + X1, family = binomial, data = dat1)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -3.79980    0.59398  -6.397 1.58e-10 ***
## E            1.87370    0.12851  14.580  < 2e-16 ***
## X1           0.08006    0.05854   1.368    0.171
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
```

```
##
##     Null deviance: 2431.7  on 4999  degrees of freedom
## Residual deviance: 2251.8  on 4997  degrees of freedom
## AIC: 2257.8
##
## Number of Fisher Scoring iterations: 5

summary(mod2)

##
## Call:
## glm(formula = D ~ E * X1, family = binomial, data = dat2)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -3.61600    0.69018  -5.239 1.61e-07 ***
## E           -0.94489    1.30453  -0.724   0.4689
## X1           0.06516    0.06825   0.955   0.3397
## E:X1         0.27752    0.12764   2.174   0.0297 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 2505.2  on 4999  degrees of freedom
## Residual deviance: 2305.7  on 4996  degrees of freedom
## AIC: 2313.7
##
## Number of Fisher Scoring iterations: 5

summary(mod3)

##
## Call:
## glm(formula = D ~ E + X1 + X2, family = binomial, data = dat3)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -6.94492    0.70991  -9.783  < 2e-16 ***
## E            1.66888    0.13097  12.742  < 2e-16 ***
## X1           0.09884    0.05766   1.714   0.0865 .
## X2           0.14988    0.01912   7.839 4.55e-15 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 2546.7  on 4999  degrees of freedom
## Residual deviance: 2342.9  on 4996  degrees of freedom
## AIC: 2350.9
```

```
##
## Number of Fisher Scoring iterations: 6
```

3. Write the analytic expression of `mod0`, `mod1`, `mod2` and `mod3`, showing the numerical values of the β coefficients estimates. Interpret $\hat{\beta}$ values.

11.3 Assessing the quality of the simulation study

To assess the quality of a simulation study, we can compare the true values of the parameters used to simulate the data with the estimated values of such parameters obtained when fitting the true model to the simulated data. If there is no bias, the distribution of the estimated parameters should be centered in the corresponding true value. Since simulated data are affected by sampling error, it would not be appropriated using estimations under a single simulated data to estimate bias. Hence, we need to perform de data simulation and model fitting process a (high) number of times in order to estimate bias. To do that, we can proceed as follows:

1. Generate $M = 100$ times the dataset `dat3` but using each time a different value for the argument `seed`. Store the M datasets in the *list* `dat3list`;
2. For each of the M datasets in `dat3list`:
   (a) Fit model `mod3`;
   (b) Use the $\hat{\beta}$ values to compute and store the estimated values of `pDmuXE0`, `pDmuXE1`, `ORDeltaX1E0`, `ORDeltaX1E1`, `ORDeltaX2E0` and `ORDeltaX2E1`.
3. Use the M estimates for each of the previous measures to get the mean value and the empirical 95% confidence interval (as percentiles 2.5 and 97.5);
4. For each parameter, get empirical bias as $Bias = True\ value - Estimated\ value$;
5. Show bias graphically.

Implement the previous 5-step algorithm in two functions:

- The `getbias()` function to perform steps 1 to 4 above
- A specific `plot(x, ...)` method (i.e., `plot.getbias(x, ...)`, where `x` is an output of `getbias()`) to generate the graphical representation.

Apply `getbias()` and `plot.getbias(x, ...)` to get and visualize results. Comment.

Chapter 12

R programming* (SHS/CDA)

12.1 Correction of sleep detection

In epidemiological studies involving variables related to sleep quality, a relative economical method to estimate sleep/week status for a given study participant based on collecting data on a number of variables that are recorded by a wristwatch worn by the individual. These continuous variables are: (1) temperature on the wrist, (2) position and (3) movement of the watch, and (4) light falling on it. Usually, each of these variables are recorded by the watch as time series in 30-second epochs (i.e., one measure each 30 seconds). Once the 4 time series are downloaded from the watch to the computer, each of them is smoothed using moving average and then the 4 smoothed time series are averaged and standardized to fall into the $[0, 1]$ interval. Finally, this averaged time series is categorized as a binary variable 0/1 according to a given threshold, so that 0 means "wake" and 1 means "sleep". Hence, each 0 indicates a wake epoch and each 1 indicates a sleep epoch. For example, the segment 000000011111111111111 means that the algorithm indicates that for the first 210 seconds the individual was awake and that for the subsequent 420 seconds the individual was asleep. Note that the result of the algorithm is a prediction, it is not necessarily the truth.

According to some studies carried out in which it has been possible to compare the classification made by the algorithm (wake/sleep) with a more precise sleep detection provided by a much more expensive method (and unfeasible in many studies), the algorithm tends to overestimate sleep periods. To correct this, in some studies the so-called Webster's rules are applied:

"*These rules were: (a) after at least 4 min scored wake, the first period of 1 min scored sleep is rescored wake; (b) after at least 10 min scored wake, the first 3 min scored sleep are rescored wake; (c) after at least 15 min scored wake, the first 4 min scored sleep are rescored wake; (d) 6 min or less scored sleep surrounded by at least 10 min (before and after) scored wake are rescored wake; and (e) 10 min or less scored sleep surrounded by at least 20 min (before and after) scored wake are rescored wake.*" [9]

12.1.1 Writing R functions

1. Write a function `webster1()`, which applies Webster's rule (a). The arguments of `webster1()` must be:

   - `x`: a numeric vector with values 0/1. It corresponds to the times series for status wake/sleep, respectively, given by the algorithm, which we want to correct. It is assumed that `x` is completely observed (i.e., no `NA`s).

- `epoch`: time length, in seconds, of each epoch. If not indicated by the user, it must be 30 seconds.

The output of `webster1()` must be a 0/1 vector (of the same length as `x`), which corresponds to the corrected time series.

2. Analogously, write functions `webster2()`, `webster3()`, `webster4()` and `webster5()`, for Webster's rules (b), (c), (d) and (e), respectively.

3. Write a function `webster()`, which sequentially applies any combination of the Webster's rules. The arguments of `webster()` must be:

   - `x`: as described above.
   - `epoch`: as described above.
   - `rules`: the rules to be sequentially applied and its order. Examples: `rules = 3` means applying only rule (c); `rules = c(1, 4)` means applying rule (d) to the result of applying rule (a). If not indicated by the user, all five rules (a) to (e) must be sequentially applied.

12.1.2 Exploring computation time

The aim of this section is to estimate the computation time when using your `webster()` function and to explore how it varies when increasing the length of `x`. To do that, apply your `webster()` function with the default setting `epoch = 30` and `rules = 1:5`. For `x`, use increasing lengths corresponding to one day (i.e., length of `x` equal to 24 hours $\times$ 3600 sec/hour $\times$ 1 epoch / 30 sec = 2880 epochs), 2 days, 3 days, ... Then, provide a visualization of the relationship between `length(x)` (in days) and the computation time.

12.2 Fast categorization in a particular case

We want to compute the value of variable Y as a function of the variable X ($X \in \mathbb{R}_+$), according to equation (12.1).

$$Y = \begin{cases} 5 & \text{if } X < 10000 \\ 10 & \text{if } 10000 \leqslant X < 20000 \\ 15 & \text{if } 20000 \leqslant X < 30000 \\ 20 & \text{if } X \geqslant 30000 \end{cases} . \tag{12.1}$$

In general, X is a numeric vector of length `n` (so Y is also a numeric vector of length `n`).

12.2.1 Writing R functions

1. Implement equation (12.1) in the following functions:

   - `f1(x)`, using an `if-else` approach;
   - `f2(x)`, using the `cut()` function;
   - `f3(x)`, using your own approach (as simple as possible) but without using `if-else`, the `apply` family (or alike), parallelization, or any other complex (or wrapped) R function.

2. Use several X vectors with `n = 10000` to check that all `f1(x = X)`, `f2(x = X)`, and `f3(x = X)`, give the same result.

12.2.2 Comparing computation times

When comparing computation times, both computational noise and system variability should be accounted for. This can be done in R with the `microbenchmark()` function using its `times` argument. `microbenchmark()` executes `f1()`, `f2()`, and `f3()` the specified number of times, providing robust summary statistics such as the mean or median execution time. Additionally, variability can be summarized using the 2.5th and 97.5th percentiles across all iterations:

```
### simulate a X vector of length 1000:
set.seed(666)
# vector length:
n <- 10000
X <- runif(n = n, min = 0.1, max = 50000)

### use microbenchmark() to compare computation times:
library(microbenchmark)

# number of runs:
m <- 5000
res <- microbenchmark(f1 = f1(x = X),
                      f2 = f2(x = X),
                      f3 = f3(x = X),
                      times = m)
res <- as.data.frame(res)

### convert time from nanoseconds to miliseconds:
res$time <- res$time / 1e6

### summary: mean, median, and 2.5th and 97.5th percentiles:
res_tab <- aggregate(time ~ expr,
                     data = res,
                     FUN = function(x) {
                       c(mean   = mean(x),
                         median = median(x),
                         p2.5   = quantile(x, 0.025),
                         p97.5  = quantile(x, 0.975))
                       })
# convert the aggregate() structure to data.frame:
res_tab <- do.call(what = "data.frame", args = res_tab)
names(res_tab) <- c("function", "mean", "median", "2.5%", "97.5%")

res_tab

##   function      mean    median      2.5%     97.5%
## 1       f1 0.4509913 0.3960305 0.3450684 0.5496204
## 2       f2 1.0775141 1.0240540 0.9019748 1.4452237
## 3       f3 0.2431637 0.1270050 0.1119068 0.1869853
```

Apply the R chunk above to different lengths of X. Then, create a visualization of the computation time (mean, median, and 2.5th and 97.5th percentiles) vs the length of X. According to that visualization, does any of the three approaches perform better than the others?

Bibliography

[1] M. Guxens, MA. Mendez, J. Julvez, E. Plana, J. Forns, X. Basagaña, M. Torrent, and J. Sunyer. Cognitive function and overweight in preschool children. *American Journal of Epidemiology*, 170(4):438–446, 2009. URL https://doi.org/10.1093/aje/kwp140.

[2] K. Steindorf, M. Schmidt, S. Kropp, and J. Chang-Claude. Case-control study of physical activity and breast cancer risk among premenopausal women in Germany. *American Journal of Epidemiology*, 57(2):121–130, 2003. URL https://doi.org/10.1093/aje/kwf181.

[3] SA. Lewis, M. Antoniak, AJ. Venn, L. Davies, A. Goodwin, N. Salfield, J. Britton, and AW. Fogarty. Secondhand smoke, dietary fruit intake, road traffic exposures, and the prevalence of asthma: a cross-sectional study in young children. *American Journal of Epidemiology*, 161(5):406–411, 2005. URL https://doi.org/10.1093/aje/kwi059.

[4] J. Hoddinott, JA. Maluccio, JR. Behrman, R. Flores, and R. Martorell. Effect of a nutrition intervention during early childhood on economic productivity in Guatemalan adults. *Lancet*, 371(9610):411–416, 2008. URL https://doi.org/10.1016/S0140-6736(08)60205-6.

[5] B. Woolf. On estimating the relation between blood group and disease. *Annals of human genetics*, 19(4):251–253, 1955. URL https://doi.org/10.1111/j.1469-1809.1955.tb01348.x.

[6] NE. Breslow and NE. Day. *Statistical Methods in Cancer Research Volume I: The Analysis of Case-Control Studies*. Number 32 in IARC Scientific Publication. International Agency for Research on Cancer, Lyon, France, 1980.

[7] OS. Miettinen. Estimation of relative risk from individually matched series. *Biometrics*, 26 (1):75–86, 1970. URL https://doi.org/10.2307/2529046.

[8] G. van Belle. *Statistical Rules of Thumb (2nd Edition)*. Wiley Series in Probability and Statistics. John Wiley & Sons, Inc., Hoboken, New Jersey, USA, 2008.

[9] JB. Webster, DF. Kripke, S. Messin, DJ. Mullaney, and G. Wyborney. An activity-based sleep monitor system for ambulatory use. *Sleep*, 5(4):389–399, 1982.

Appendix A

R code for Section 7.1.1

```
compareEstimates <- function(n, p) {
  Xn <- rbinom(n = 1, size = n, prob = p)
  aproxIC <- as.numeric(prop.test(x = Xn, n = n, conf.level = 0.95)$conf.int)
  exactIC <- as.numeric(binom.test(x = Xn, n = n, conf.level = 0.95)$conf.int)
  res <- c(n = n, p = p, aproxIC, exactIC)
  names(res)[c(3, 5)] <- paste0("lower", c("Approx", "Exact"))
  names(res)[c(4, 6)] <- paste0("upper", c("Approx", "Exact"))
  return(res)
 }
```

```
nValues <- seq(10, 100, by = 10)
pValues <- c(0.01, 0.05, 0.1, 0.2, 0.5, 0.8, 0.9, 0.95, 0.99)
res <- expand.grid(n = nValues, p = pValues)
aux <- data.frame(matrix(nrow = nrow(res), ncol = 4))
auxnames <-  c("lowerApprox", "upperApprox", "lowerExact", "upperExact")
names(aux) <- auxnames
for (i in 1:nrow(aux))
 aux[i, ] <- compareEstimates(n = res$n[i], p = res$p[i])[auxnames]
myResults <- cbind(res, aux)
```

```
par(mfrow = c(3, 3), las = 1, cex.axis = 0.8)
xmin <- min(nValues)
xmax <- max(nValues)
ymin <- min(myResults[, c("lowerApprox", "lowerExact")])
ymax <- max(myResults[, c("upperApprox", "upperExact")])
delta <- 0.5
legendPos <- c(rep("topright", 5), rep("bottomright", 4))
for (i in 1:length(pValues)) {
  aux <- myResults[myResults$p == pValues[i], -2]
  ymin <- min(aux[, c("lowerApprox", "lowerExact")])
  ymax <- max(aux[, c("upperApprox", "upperExact")])
  plot(c(xmin, xmax), c(pValues[i], pValues[i]), type = "l",
       main = paste("p =", pValues[i]), ylim = c(ymin, ymax),
       xlab = "n", ylab = expression(hat(p)), lty = 2, col = "blue")
  segments(aux$n - delta, aux$lowerApprox, aux$n - delta, aux$upperApprox,
           col = "red", lwd = 2)
```

```
  segments(aux$n + delta, aux$lowerExact, aux$n + delta, aux$upperExact,
           col = "forestgreen", lwd = 2)
  abline(h = 0, lty = 3)
  abline(h = 1, lty = 3)
  legend(legendPos[i], c("Approx.", "Exact"), col = c("red", "forestgreen"),
         lwd = 2, bty = "n", cex = 0.8)
}
```

Appendix B

Estimation of OR under a 1:1 matching case-control design

In this Appendix, we provide a proof of equation (9.1).

Step 1. Define the exposure probabilities. Let $D = 1$ denote a case and $D = 0$ denote a control. Likewise, let $E = 1$ denote exposure and $E = 0$ denote no exposure.

Define the exposure probability among cases,

$$p_1 \stackrel{\text{def}}{=} P(E = 1 \mid D = 1), \tag{B.1}$$

and the exposure probability among controls,

$$p_0 \stackrel{\text{def}}{=} P(E = 1 \mid D = 0). \tag{B.2}$$

Assume that, conditional on the matching variables (or equivalently, within a matched stratum), the exposure status of the case and the control are independent. Under this assumption, the joint probabilities of the four possible matched-pair configurations in Table 9.2 are obtained by multiplying the corresponding marginal probabilities.

Step 2. Compute the probability of each pair type. There are four possible exposure patterns for a matched case-control pair.

Type 1 (both exposed):

$$P(\text{Type 1}) = p_1 p_0. \tag{B.3}$$

Type 2 (case exposed, control unexposed):

$$P(\text{Type 2}) = p_1(1 - p_0). \tag{B.4}$$

Type 3 (case unexposed, control exposed):

$$P(\text{Type 3}) = (1 - p_1)p_0. \tag{B.5}$$

Type 4 (both unexposed):

$$P(\text{Type 4}) = (1 - p_1)(1 - p_0). \tag{B.6}$$

Obviously, the four probabilities given in (B.3)–(B.6) sum to one.

Step 3. Restrict attention to discordant pairs. A matched pair is said to be *discordant* if exactly one member of the pair is exposed. Therefore,

$$\{\text{Discordant}\} = \{\text{Type 2}\} \cup \{\text{Type 3}\},$$

and, using (B.4) and (B.5),

$$P(\text{Discordant}) = p_1(1 - p_0) + (1 - p_1)p_0. \tag{B.7}$$

Step 4. Apply the definition of conditional probability. The event "the case is exposed" within a discordant pair is precisely the event that the pair is of Type 2. Hence,

$$P(\text{Case exposed} \mid \text{Discordant}) = \frac{P(\text{Type 2})}{P(\text{Discordant})}. \tag{B.8}$$

Substituting (B.4) and (B.7) into (B.8) yields

$$P(\text{Case exposed} \mid \text{Discordant}) = \frac{p_1(1 - p_0)}{p_1(1 - p_0) + (1 - p_1)p_0}. \tag{B.9}$$

Equation (B.9) is the first equality shown in the equation (9.1).

Step 5. Express the conditional probability in terms of the odds ratio. By definition, the odds ratio comparing the odds of exposure among cases and controls is

$$\text{OR} = \frac{\dfrac{p_1}{1 - p_1}}{\dfrac{p_0}{1 - p_0}} = \frac{p_1(1 - p_0)}{(1 - p_1)p_0}. \tag{B.10}$$

Equation B.10 provides the equality $p_1(1-p_0) = \text{OR}(1-p_1)p_0$, which substituted in equation (B.9), provides

$$P(\text{Case exposed} \mid \text{Discordant}) = \frac{\text{OR}(1 - p_1)p_0}{\text{OR}(1 - p_1)p_0 + (1 - p_1)p_0} = \frac{\text{OR}}{\text{OR} + 1}. \tag{B.11}$$

Equation (B.11) is the second equality shown in the equation (9.1).

Appendix C

R code for Chapter 10

Auxiliar function `multinomialSample`, which is called by `estimateRisks` function:

```
##############################################################################
##############################################################################
###
### 'multinomialSample': auxiliar function to simulate a random sample
###                      from a multivariate distribution.
###
##############################################################################
###
### Input:
###
### n:     sample size
### probs: vector of length 'l' number proportional to the probabilities
### seed:  seed for results reproducibility
###
##############################################################################
###
### Output:
###
### vector of length 'l' with the frequencies
###
###
multinomialSample <- function(n, probs, seed = NULL) {
  if (!is.null(seed))
    set.seed(seed)
  l <- length(probs)
  res <- rep(0, l)
  names(res) <- 1:l
  samp <- rep(1:length(probs), rmultinom(1, size = n, probs))
  aux <- table(samp)
  res[names(aux)] <- aux
  return(res)
}
##############################################################################
###
### END OF  'multinomialSample' FUNCTION
###
##############################################################################
```

`estimateRisks` function to be complete (indicated with `### TO COMPLETE`):

```
##############################################################################
##############################################################################
###
### 'estimateRisks': function to simulate a 2x2 contingency table under three
### different sampling designs and estimate bias of its probabilities
###
##############################################################################
###
### Input:
###
### n:     sample size
### piE:   prevalence of the exposure E in the population
### pi0:   prevalence of the disease D among the non exposed in the population
### pi1:   prevalence of the disease D among the exposed in the population
### propE: proportion of exposed individuals in the sampling
### propD: proportion of cases in the sampling
### seed: seed for results reproducibility
###
##############################################################################
###
### Output:
###
### vector of length 'l' with the frequencies
###
###
estimateRisks <- function(n, piE, pi0, pi1, propE, propD, seed = NULL) {
  # Notation:
  #    D:  Case
  #    ND: Control
  #    E: Exposed
  #    NE: Non exposed
  #    a: contingency table [1, 1]
  #    b: contingency table [1, 2]
  #    c: contingency table [2, 1]
  #    d: contingency table [2, 2]

  # Fix the seed if any:
  if (!is.null(seed))
    set.seed(seed)

  # List for the contingency tables:
  tabs <- vector("list", 3)

  # Sampling design 1:
  # rows: 1 (NE); 2 (E)
  # columns: 1 (ND); 2 (D)
  # Compute the join probabilities:
  piTab1 <- (1 - piE) * (1 - pi0)
```

```r
piTab2 <- ### TO COMPLETE
piTab3 <- ### TO COMPLETE
piTab4 <- ### TO COMPLETE
piTab <- c(piTab1, piTab2, piTab3, piTab4)
# Simulate the sample:
tab <- multinomialSample(n = n, probs = piTab)
tab <- matrix(tab, nrow = 2, ncol = 2, byrow = TRUE)
# Add margins:
tab <- addmargins(tab)
rownames(tab) <- c("non E", "E", "Total")
colnames(tab) <- c("non D", "D", "Total")
# Save table:
tabs[[1]] <- tab

# Sampling design 2:
# rows: 1 (NE); 2 (E)
# columns: 1 (ND); 2 (D)
# Simulate row 1:
nE <- round(n * propE)
row1col2 <- ### TO COMPLETE
row1col1 <- ### TO COMPLETE
# Simulate row 2:
row2col2 <- ### TO COMPLETE
row2col1 <- ### TO COMPLETE
tab <- matrix(c(row1col1, row1col2, row2col1, row2col2),
              nrow = 2, ncol = 2, byrow = TRUE)
# Add margins:
tab <- addmargins(tab)
rownames(tab) <- c("non E", "E", "Total")
colnames(tab) <- c("non D", "D", "Total")
# Save table:
tabs[[2]] <- tab

# Sampling design 3:
# rows: 1 (ND); 2 (D)
# columns: 1 (NE); 2 (E)
# Simulate row 1:
nD <- round(n * propD)
piD <- pi0 + (pi1 - pi0) * piE
row1col2 <- ### TO COMPLETE
row1col1 <- ### TO COMPLETE
# Simulate row 2:
row2col2 <- ### TO COMPLETE
row2col1 <- ### TO COMPLETE
tab <- matrix(c(row1col1, row1col2, row2col1, row2col2),
              nrow = 2, ncol = 2, byrow = TRUE)
# Add marginals:
tab <- addmargins(tab)
rownames(tab) <- c("non D", "D", "Total")
colnames(tab) <- c("non E", "E", "Total")
```

```
  # Save table:
  tabs[[3]] <- tab

  # Label tables:
  names(tabs) <- paste0("Design", 1:3)

  # Summary table for the estimates:
  est <- matrix(nrow = 4, ncol = 6)
  rownames(est) <- c(names(tabs), "Population")
  colnames(est) <- c("P(E)", "P(D)", "P(D|nonE)", "P(D|E)","P(E|nonD)","P(E|D)")
  for (i in 1:2) {
    est[i, 1] <- tabs[[i]][2, 3] / n
    est[i, 2] <- tabs[[i]][3, 2] / n
    est[i, 3] <- tabs[[i]][1, 2] / tabs[[i]][1, 3]
    est[i, 4] <- tabs[[i]][2, 2] / tabs[[i]][2, 3]
    est[i, 5] <- tabs[[i]][2, 1] / tabs[[i]][3, 1]
    est[i, 6] <- tabs[[i]][2, 2] / tabs[[i]][3, 2]
   }
  i <- 3
  est[i, 1] <- tabs[[i]][3, 2] / n
  est[i, 2] <- tabs[[i]][2, 3] / n
  est[i, 3] <- tabs[[i]][2, 1] / tabs[[i]][3, 1]
  est[i, 4] <- tabs[[i]][2, 2] / tabs[[i]][3, 2]
  est[i, 5] <- tabs[[i]][1, 2] / tabs[[i]][1, 3]
  est[i, 6] <- tabs[[i]][2, 2] / tabs[[i]][2, 3]
  est[4, ] <- c(piE, piD, pi0, pi1, (1 - pi1) * piE / (1 - piD), pi1 * piE / piD)

  # Bias (in percentage):
  pop <- matrix(rep(est[4, ], 3), nrow = length(tabs), ncol = ncol(est),
                byrow = TRUE)
  bias100 <- round(100 * (est[1:length(tabs), ] / pop - 1), 1)

  # Result:
  res <- list(tables2x2 = tabs, estimates = est, percentBias = bias100)
  return(res)
 }
##############################################################################
###
### END OF  'estimateRisks' FUNCTION
###
##############################################################################
##############################################################################
```